

2026 年 7 月 3 日 Version 1.6

---

# ハッカソン ～計画書と設計書～

---

## チーム 26

24G1040 : 金枝 優介

24G1074 : 荘司 あかね

24G1125 : 三芝 空音

25G1009 : 飯塚 蓮

25G1034 : 恩田 隼士

25G1063 : 佐藤 羽琉

# 目次

|       |                        |    |
|-------|------------------------|----|
| 第 1 章 | プロジェクトの計画書             | 1  |
| 1.1   | プロジェクトの概要              | 1  |
| 1.1.1 | プロジェクトの題目              | 1  |
| 1.1.2 | プロジェクトの目的              | 1  |
| 1.2   | 社会的背景                  | 1  |
| 1.2.1 | 既存の技術とプロジェクトの独自性       | 2  |
| 1.3   | スコープ・マネジメント            | 4  |
| 1.3.1 | 成果物                    | 4  |
| 1.3.2 | プロジェクトの対象範囲と非対象範囲      | 4  |
| 1.3.3 | 機能分解構造 FBS             | 5  |
| 1.3.4 | 製品分解構造 PBS             | 5  |
| 1.3.5 | FBS と PBS の対応関係        | 6  |
| 1.4   | 作業計画                   | 7  |
| 1.4.1 | 作業分解構造 WBS と管理番号       | 7  |
| 1.4.2 | アローダイアグラムとクリティカルパス     | 9  |
| 1.4.3 | 役割分担                   | 9  |
| 1.4.4 | ガントチャート                | 10 |
| 1.5   | 検証と妥当性確認: V&V 計画       | 11 |
| 1.5.1 | プロジェクトの成果物に対する有効指標 MOE | 11 |
| 1.5.2 | 有効指標 MOE に対応する性能指標 MOP | 12 |
| 1.5.3 | システムテストで評価する性能指標 MOP   | 12 |
| 1.5.4 | 結合テストで評価する性能指標 MOP     | 13 |
| 1.5.5 | 単体テストで評価する性能指標 MOP     | 13 |
| 1.5.6 | プロジェクト中に監視する技術性能指標 TPM | 16 |
| 1.6   | 資源・調達管理                | 16 |
| 1.7   | リスク管理                  | 18 |
| 第 2 章 | システムの基本設計              | 19 |

|       |                   |    |
|-------|-------------------|----|
| 2.1   | 機能一覧              | 19 |
| 2.2   | システム構成図           | 21 |
| 2.3   | 操作インターフェース設計      | 21 |
| 2.4   | 状態遷移図             | 23 |
| 第3章   | システムの詳細設計         | 25 |
| 3.1   | 部品一覧              | 25 |
| 3.2   | ハードウェア設計          | 27 |
| 3.3   | ソフトウェア設計          | 29 |
| 3.3.1 | ファイル構成            | 29 |
| 3.3.2 | 通信設計              | 29 |
| 3.3.3 | LED マトリクスへの歌詞表示   | 31 |
| 3.3.4 | 音生成・再生            | 33 |
| 3.3.5 | 受光機能              | 33 |
| 3.4   | スイッチによる入力制御の詳細設計  | 37 |
| 3.4.1 | 入力ピンの割り当てと初期設定    | 37 |
| 3.4.2 | 可変抵抗によるテンポ入力      | 38 |
| 3.4.3 | 可変抵抗による分圧の原理      | 38 |
| 3.4.4 | タクトスイッチによる起動・停止制御 | 39 |
| 3.4.5 | 入力制御の処理フロー        | 41 |
| 3.4.6 | オブジェクト設計 (クラス図)   | 42 |
| 3.4.7 | 関数設計              | 44 |
| 3.5   | 処理フローの設計          | 45 |
| 3.6   | テストの設計            | 50 |
| 3.6.1 | システムテストの設計        | 50 |
| 3.6.2 | 結合テストの設計          | 51 |
| 3.6.3 | 単体テストの設計          | 51 |

# 第 1 章 プロジェクトの計画書

## 1.1 プロジェクトの概要

### 1.1.1 プロジェクトの題目

本プロジェクトの題目は、「単一のフルカラー LED と Arduino を用いた光無線通信による楽器間同期演奏システムの開発」である。

### 1.1.2 プロジェクトの目的

従来の同期システムでは、有線接続による配線制約や、無線通信における通信遅延・通信環境依存などが課題となっている。本プロジェクトの目的は、LED の光を用いた無線通信による楽器間同期演奏システムを構築し、楽器演奏のズレを最小限に抑え、合奏を成立させることである。さらに、利用者が楽器変更やテンポ変更等の演奏機能を制御するための Web サイトや視覚的に合奏を認識するために LED マトリクス歌詞表示システムを構築することでシステムの操作性及びエンターテインメント性の向上を目指す。最終的なゴールは、ユーザの操作に応じて、複数楽器の同期演奏を成立させることである。

## 1.2 社会的背景

近年、音楽演奏や視覚演出の分野において、出力装置を複数台組み合わせた作品やシステムが研究されている。例えば神田らは複数のモバイルデバイスを用いたインタラクティブメディアアートを提案している [1]。これらのシステムでは、各装置が正確なタイミングで動作することが求められ、リアルタイム同期処理が重要な要素となる。

特に音楽演奏においては、複数の演奏者や演奏装置が同一テンポで動作する必要がある。わずかな同期ズレであっても演奏品質に影響を与える。音声遅延が演奏に与え

る影響は大きく、先行研究では数 ms 程度の遅延であっても演奏品質や演奏感覚に影響を与えることが報告されている [2]。そのため、演奏同期システムにおいては、演奏に支障を与えるような遅延や同期ズレを低減するための高精度な同期制御技術が求められる。

また、近年の IoT 技術の発展により、様々なデバイスがネットワークに接続されるようになり、データ流通量のさらなる増加へとつながっている [3]。このような技術は、スマートホームや産業機器の制御、エンターテインメントシステムなど、多様な分野において応用されている。

一方で、複数装置を連携させる従来の同期システムでは、有線接続による配線制約や、無線通信における通信遅延・通信環境依存などの課題が存在する。そのため、複数装置を柔軟に同期可能な、新たな同期手法が求められている。

また、輪唱のような複数パートが異なるタイミングで演奏する楽曲では、聴衆が各パートの演奏状況を聴覚のみで把握することは難しい。音楽演奏において視覚情報と聴覚情報を組み合わせることで、聴衆の楽曲理解や没入感が向上することが報告されており [4]、演奏システムにおける視覚的演出の重要性は高まっていると言える。

### 1.2.1 既存の技術とプロジェクトの独自性

既存の通信方法として現在幅広く利用されているものに、有線通信および Bluetooth や Wi-Fi を用いた無線通信がある。

有線通信は通信の安定性が高く遅延の影響を受けにくいという利点を持つが、複数装置を接続するためには配線が必要となり、設置の自由度が低下するという課題がある。特に近距離環境での複数装置の同期において、配線の取り回しがシステム構成の複雑化を招く。一方、無線通信は配線を必要とせず柔軟なシステム構成が可能であるが、電波干渉や通信環境の影響を受けやすく、遅延やパケット損失が発生する可能性がある [5]。

また、本システムに関連する技術として光無線通信が挙げられる。光無線通信とは、我々が日常的に使用している放送や携帯電話の電波と同じ電磁波の一種である光を電波媒体とするシステムである [6]。この通信システムの光源には LED や LD、VCSEL が用いられ、光信号の受信用検出器には Pin-PD や APD が使われる。

従来の光通信では、光の有無や強弱といった単一の情報を扱う方式が一般的であり、伝達可能な情報量が限られるという課題がある。これは、従来の強度検出型の光通信方式では、主に光の強弱のみを情報として利用していることに起因する。また、

外乱光の影響を受けやすく、環境光の変動による誤検出が生じる可能性がある。さらに、複数の信号を識別することが困難であり、多数の装置を同時に制御する用途には適していない。

これらの課題に対し、本システムは近距離環境での複数装置の同期制御を対象とし、フルカラー LED とカラーセンサを用いた光通信を採用することで配線を不要とし、近距離環境における配線削減およびシステム構成の簡略化を図る。

単一の LED から全演奏者へ同時に信号を送信する構成とすることで、演奏者間の信号のばらつきをなくし、同期ズレを防ぐ設計とした。光は電波と比較して外部からの干渉を受けにくく、混線が生じないため、無線通信で問題となる同期ズレの低減が期待される。また、光通信ではネットワークを介した無線通信と異なり、パケット損失や通信環境変動の影響を受けにくく、安定した信号伝達を行うことができると考えられる。加えて、単一の LED を使用することで、有線通信のように装置の増加に伴って分配器やケーブルが増大する問題がなく、受光可能範囲内であれば装置の追加や配置変更が容易である。さらに、光通信はネットワークを介した無線通信と比較して、通信処理が単純であるため、低遅延化が期待できる。加えて、センサ周辺を遮光することで外部照明の影響を低減し、誤検出の低減を図る。

本システムではフルカラー LED とカラーセンサを用いることで、光の色、発光間隔、光強度といった複数のパラメータを同時に利用した通信を実現している。これにより、従来の強度ベースの光通信と比較してより多くの情報を表現可能である。さらに、光の点灯間隔をセンサで検出することでテンポ情報を直接取得し、数値データを用いずに演奏タイミングを制御する点に特徴がある。また、色ごとに異なる意味を割り当てることで複数の楽器や制御信号を同時に扱うことができる。

以上より、本システムは光通信の簡易性と多機能性を両立した光同期システムである点に独自性がある。さらに、本システムでは LED マトリクスによる歌詞表示機能を実装することで、輪唱における各パートの演奏タイミングを視覚的に提示し、聴覚情報のみでは把握しにくい輪唱の構造を視覚情報と組み合わせて表現する点にも独自性がある。

## 1.3 スコープ・マネジメント

### 1.3.1 成果物

本プロジェクトにおける最低限達成すべき機能として、光無線通信による楽器間の演奏ズレが限りなく少ない合奏機能を実装する。同期通信、演奏制御、エンターテイメントとしての歌詞表示を含めた本プロジェクトの成果物は、以下の通りである。

- フルカラー LED を用いた光通信による同期演奏システム
- 各楽器の音再生プログラム
- LED マトリクスによる歌詞表示機能
- Web 演奏制御機能

### 1.3.2 プロジェクトの対象範囲と非対象範囲

フルカラー LED を用いた同期演奏システムを構築するために必要な本プロジェクトの対象範囲は以下の通りである。

- LED による光通信機能実装
- カラーセンサを用いた光信号の受信、解析
- テンポ取得
- 4 種類の楽器音再生
- Web からのテンポ変更、楽器変更、音量変更、楽譜選択・ループ演奏選択機能実装
- LED マトリクスによる歌詞表示

一方で、本プロジェクトでは、指揮者、演奏者間の同期通信に焦点を当てるため、実際の楽器を演奏するのではなくデータとして楽器を演奏する。また、Web サイトでの演奏制御は同期処理に負荷をかけないように、演奏前に楽器の変更などを行うためネットワーク環境の整備は本プロジェクトでは対象外とした。よって、非対象範囲は以下の通りである。

- 商用レベルの高速・長距離光通信

- 実際の楽器を演奏するための入力インターフェース
- システム運用に必要なネットワーク環境の整備

### 1.3.3 機能分解構造 FBS

本システムのユーザ目線から見た機能分解としての FBS を図 1.1 に示す。ユーザが可能な制御は演奏中と、演奏前の二つに分けられる。演奏前は、演奏者ごとに楽器変更、楽譜選択、個別音量の調節が可能で、演奏中は、テンポ変更、全体音量の調節、およびループ演奏の切り替えが可能である。また、それらの演奏制御は物理的なスイッチ以外に、Web サイトを通じて行うことができる。エンターテインメントとしては、各楽器用の Arduino の LED マトリクスに演奏中の歌詞を表示する。

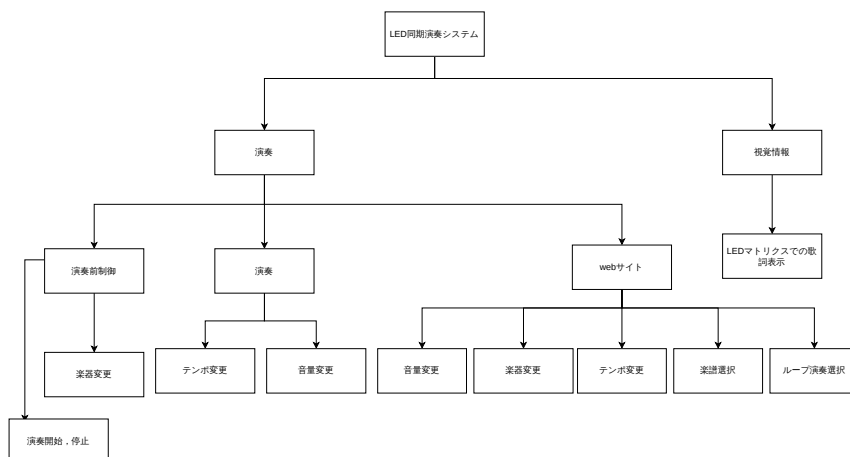


図 1.1 機能分解構造 (FBS)

### 1.3.4 製品分解構造 PBS

本システムの FBS を実装するための機能である PBS を図 1.2 に示す。ハードウェアの面では、演奏同期のための LED の送受信の機能と、演奏開始や停止などの演奏制御をボタン、スイッチを用いて実装する。また、ソフトウェアの面では、演奏を実



現するための楽器音、楽譜、歌詞のデータベース及び、制御のための web ページを実装する。

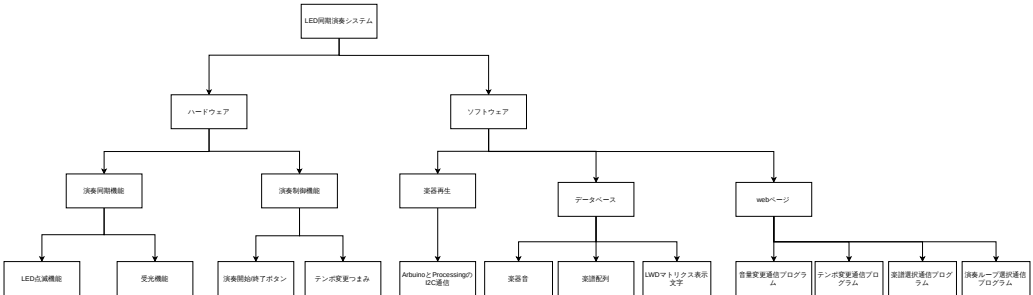


図 1.2 製品分解構造 (PBS)

### 1.3.5 FBS と PBS の対応関係

FBS と PBS の対応関係を以下の表 1.1 に示す。FBS の各機能に対して、それを実現する PBS のサブ成果物を対応付けた。各機能は 1 つ以上の成果物によって実現される。

「楽器演奏」機能については、ソフトウェア側の「演奏プログラム（ピアノ・バイオリン・フルート・パーカッション再生）」と、ハードウェア側の「音再生用 PC」によって実現される。「演奏制御」機能のうち、「演奏同期」は「送受信プログラム」と「フルカラー LED」「カラーセンサ」の組み合わせにより、光無線通信として実装される。また、「演奏開始・停止」は「Web サイト」と指揮者 Arduino に設置された「タクトスイッチ」がその役割を担う。「Web サイト」による「テンポ変更」「音量変更」「楽器変更」「楽譜選択」「ループ演奏選択」は、Web サイトに対応した各プログラムの構築により実現される。「視覚情報表示」機能である「LED マトリクス歌詞表示」は、ソフトウェア側の「LED マトリクス表示プログラム（文字配列データ）」と、ハードウェア側の「演奏者 Arduino」によって実装される。このように、本プロジェクトにおける全ての機能要件は、定義されたソフトウェアおよびハードウェアの構成要素によって網羅されている。

**表 1.1 FBS と PBS の対応関係**

| FBS (機能)      | PBS (成果物)                             |
|---------------|---------------------------------------|
| ピアノ演奏         | 演奏プログラム (ピアノ音再生), 楽器音データ, 楽譜配列データ     |
| バイオリン演奏       | 演奏プログラム (バイオリン音再生), 楽器音データ, 楽譜配列データ   |
| フルート演奏        | 演奏プログラム (フルート音再生), 楽器音データ, 楽譜配列データ    |
| パーカッション演奏     | 演奏プログラム (パーカッション音再生), 楽器音データ, 楽譜配列データ |
| 演奏開始・停止       | 送信プログラム, 受信プログラム                      |
| 演奏同期          | 送信プログラム, 受信プログラム                      |
| 楽器変更          | Web サイト (楽器変更プログラム)                   |
| テンポ変更         | Web サイト (テンポ変更プログラム), テンポ変更つまみ        |
| 音量変更          | Web サイト (音量変更通信プログラム)                 |
| 楽譜選択          | Web サイト (楽譜選択プログラム)                   |
| 演奏ループ選択       | Web サイト (演奏ループ選択プログラム)                |
| LED マトリクス歌詞表示 | LED マトリクス表示プログラム, LED マトリクス文字配列       |

## 1.4 作業計画

### 1.4.1 作業分解構造 WBS と管理番号

本開発における作業分解構造 WBS と管理番号を表 1.2 に示す。

表 1.2 本開発における WBS と管理番号

| 目的                 | フェーズ     | タスク          | 活動タスク                       | 担当者    |
|--------------------|----------|--------------|-----------------------------|--------|
| LED の発光色による光同期システム | 100 設計   | 110 基本設計     | 111 楽器演奏                    | 各楽器担当者 |
|                    |          |              | 112Web サイト                  | 金枝     |
|                    |          |              | 113LED マトリクスデザイン            | 三芝     |
|                    |          | 120 詳細設計     | 121LED 点滅機能                 | 佐藤     |
|                    |          |              | 122 受光機能                    | 飯塚     |
|                    |          |              | 123 演奏開始・停止ボタン              | 恩田     |
|                    |          |              | 124Arduino と Processing の通信 | 飯塚     |
|                    |          |              | 125 テンポ変更機能                 | 佐藤     |
|                    |          |              | 126 楽器変更機能                  | 金枝     |
|                    |          |              | 127 楽器音                     | 各楽器担当者 |
|                    |          |              | 128 楽譜配列                    | 恩田     |
|                    |          |              | 129LED マトリクス文字配列            | 三芝     |
|                    | 200 機材調達 | 210 部品選定     | 211 購入または借用                 | 全員     |
|                    | 300 実装   | 310 楽器音作成    | 311 ピアノ音                    | 佐藤     |
|                    |          |              | 312 フルート音                   | 佐藤     |
|                    |          |              | 313 バイオリン音                  | 恩田     |
|                    |          |              | 314 パーカッション音                | 荘司     |
|                    |          | 320 同期システム   | 321 光信号発信                   | 佐藤     |
|                    |          |              | 322 光信号受信                   | 飯塚     |
|                    |          |              | 323 演奏開始・停止ボタン              | 恩田     |
|                    |          |              | 324 シリアル通信 送信               | 飯塚     |
|                    |          |              | 325 シリアル通信 受信               | 飯塚     |
|                    |          | 330Web サイト作成 | 331 テンポ変更機能                 | 佐藤     |
|                    |          |              | 332 楽器変更機能                  | 金枝     |
|                    |          |              | 333 楽譜選択機能                  | 金枝     |
|                    |          |              | 334 演奏ループ選択機能               | 金枝     |
|                    |          |              | 335 音量変更機能                  | 金枝     |
|                    |          | 340 演出       | 341LED マトリクス                | 三芝     |
|                    | 400 テスト  | 410 単体テスト    | 411 楽器音テスト                  | 各楽器担当者 |
|                    |          |              | 412 開始停止スイッチテスト             | 恩田     |
|                    |          |              | 413 テンポ変更スイッチテスト            | 飯塚     |
|                    |          | 420 結合テスト    | 421 光通信の同期テスト               | 全員     |
|                    |          |              | 422Web 通信テスト                | 全員     |
|                    |          |              | 423 接続テスト                   | 全員     |
|                    |          |              | 424 遅延テスト                   | 全員     |
|                    |          |              | 425 合奏が成り立っているかのテスト         | 全員     |
|                    | 500 仕上げ  | 510 結合       | 511 全体結合                    | 全員     |
|                    |          | 520 システム評価会  | 521 システム評価会実施               | 全員     |

各タスクには 100 番台から 500 番台の管理番号を付与し、設計・機材調達・実装・テスト・仕上げの 5 フェーズで構成した。

### 1.4.2 アローダイアグラムとクリティカルパス

作成したアローダイアグラムを図 1.3 に示す。赤線で示すクリティカルパスは、111～113（基本設計）、121～129（詳細設計）、311～325（実装）、411～413（単体テスト）、421～425（結合テスト）、511～521（仕上げと結合）であり、総所要日数は 49 日である。

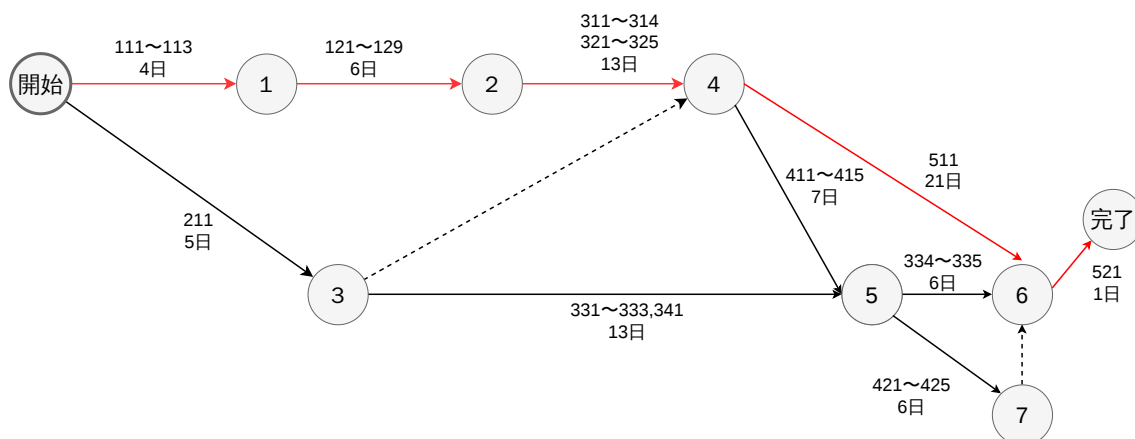


図 1.3 本開発のアローダイアグラム

### 1.4.3 役割分担

各メンバーの役割分担は表 1.2 に示す通りである。分担割り当ては、LED 点滅機能や受光機能、楽器演奏機能などの本プロジェクトで最低限必要な機能の設計は 2 年生の飯塚、佐藤、恩田が行う。また、LED マトリクス歌詞表示機能や Web サイトでの楽器変更、テンポ変更、音量変更、楽譜選択、ループ演奏選択などの次に必要とする

機能の設計を 3 年生全体で行う。結合テストおよび機材調達については全員で対応する。

#### 1.4.4 ガントチャート

本プロジェクトにおけるガントチャートを図 1.4 に示す。横軸に日付、縦軸に WBS の各タスクを示す。プロジェクトは 2026 年 5 月 20 日に開始し、設計・機材調達・実装・テストの順に進める。設計フェーズを 5 月中に完了させ、6 月上旬までに実装、7 月頭までにテストを完了することを目標とする。テスト終了予定日から 1 週間は、不具合発生時の修正作業および開発遅延への対応を行うための予備期間として確保する。これにより、リスク管理で想定する遅延や同期精度の問題に対して柔軟に対応できるようにする。また、中間テストのある 6 月上旬に、土日を含めた 3 日間をテスト対策期間として空けている。

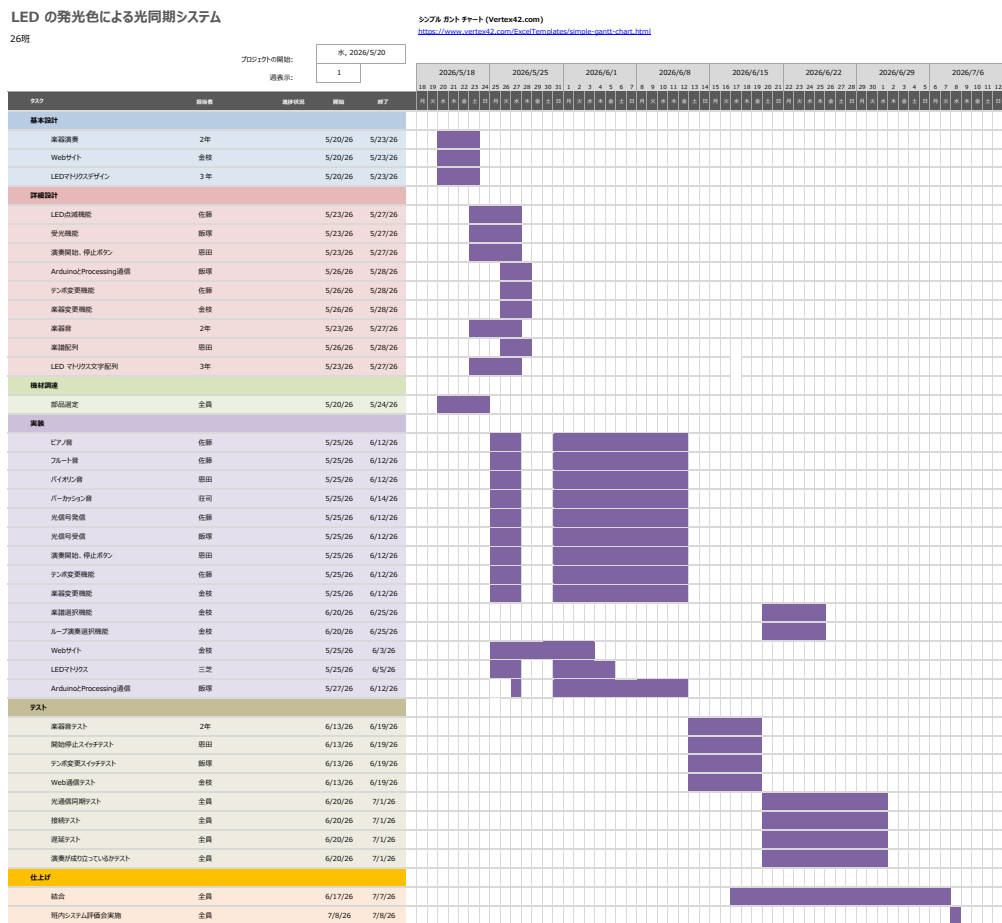


図 1.4 ガントチャート

## 1.5 検証と妥当性確認: V&V 計画

### 1.5.1 プロジェクトの成果物に対する有効指標 MOE

本プロジェクトの成果物は、離れた場所にいる演奏者が指揮者の LED による光信号を基準として同期しながら合奏できるシステムである。このシステムの有効性を評価するための有効指標（MOE）を以下に示す。

- MOE1：光同期の実現

指揮者の光信号が各演奏者に遅延なく届き、演奏者間の同期がとれた合奏が成立

すること。

- **MOE2：音色の再現性**

本システムが出力する楽器音が、実際の楽器音を十分な精度で再現していること。

- **MOE3：Web サイトによるリアルタイム制御**

指揮者が Web サイトを通じてテンポや楽器などの演奏パラメータをリアルタイムに変更できること。

## 1.5.2 有効指標 MOE に対応する性能指標 MOP

各 MOE に対応する定量的な性能指標（MOP）を以下に示す。

表 1.3 MOE と MOP の対応

| MOE  | 評価項目         | MOP              | 目標値       |
|------|--------------|------------------|-----------|
| MOE1 | 光と音の発生時間差    | 通信遅延             | 125 ms 以内 |
|      | 演奏者間の同期ズレ    | 楽器間同期ズレ          | 40 ms 以内  |
|      | 光同期の通信精度     | パケット損失率（IPLR）    | 1%以内      |
| MOE2 | 楽器音の再現度      | 立ち上がり時間差（AT 差）   | 60 ms 以内  |
|      |              | スペクトル重心相対差（SC 差） | 24%以内     |
|      |              | スペクトル変動度差（DSV 差） | 16%以内     |
|      |              | 奇偶倍音比差（OER 差）    | 0.20 以内   |
|      |              | 音色アンケート評価        | 3.51 以上   |
| MOE3 | Web サイトの応答速度 | 送信～反映までの遅延       | 100 ms 以内 |
|      | Web サイトの通信精度 | パケット損失率（IPLR）    | 1%以内      |

## 1.5.3 システムテストで評価する性能指標 MOP

システムテストでは、システム全体を結合した状態で以下の MOP を評価する。

- **光と音の発生時間差（目標：125 ms 以内）**

指揮者の LED が発光してから演奏者 PC のスピーカーから音が出るまでの時間差を評価する。光（映像）が先、音（音声）が後の場合、125 ms を超えるとズレに気づき、185 ms を超えると許容できなくなることが知られている [7]。

- **楽器間の同期ズレ（目標：40 ms 以内）**

複数の演奏者間における各拍の発音タイミングのズレを評価する。先行音と後続音のズレが 40 ms 以内であれば一つの音として認識される [8].

- **Web サイト送信～演奏反映までの遅延（目標：100 ms 以内）**

Web サイトでテンポなどのパラメータを Web サイトで変更してから、演奏プログラムに反映されるまでの時間を評価する。人間がシステムを操作した際、100 ms 以内の応答であれば遅延を知覚しない [9].

#### 1.5.4 結合テストで評価する性能指標 MOP

結合テストでは、各機能モジュールを接続した状態で以下の MOP を評価する。

- **光同期のパケット損失率（目標：1%以内）**

指揮者から演奏者への光同期通信において、LED の発光を光センサで検知できたかを計数し、パケット損失率を評価する。ITU-T の規定するリアルタイムデータ通信の許容 IPLR は 1%以下である [10].

- **Web サイトのパケット損失率（目標：1%以内）**

Web サイトから演奏プログラムへの制御コマンド送信において、送信ログと受信ログを比較し、パケット損失率を評価する。同様に ITU-T の規定する許容 IPLR である 1%以下を目標とする [10].

#### 1.5.5 単体テストで評価する性能指標 MOP

単体テストでは、個々の機能モジュール単体で以下の MOP を評価する。

- **音色の再現性（目標：各音色特徴量が目標差以内）**

実際の楽器音の録音データと、本システムが出力した音の録音データを録音して比較する。

人間の音色知覚を構成する特徴量である以下の 4 つを用いて音色の再現精度を評価する [17]. 比較の前処理として、両者の音量を RMS 正規化により統一し、pYIN アルゴリズムで推定した F0 を用いて音程が一致していることを確認する。AT・SC・DSV・OER の 4 つの音色特徴量を算出し、各音程ごとに参照音源と再現音のズレ量（または相対差）を求め、全音程にわたる平均値を目標値と比較する。それぞれの数値の説明は以下に示す。本テストは各楽器・音程ごとに行った



うえで平均を取り評価する。

**立ち上がり時間 (AT)：**RMS エンベロープがピーク値に到達するまでの時間 [ms]。音の立ち上がり（アタック）の速さが参照音とどれだけズレているかを評価する。音声波形から RMS エンベロープ（短時間ごとの RMS 振幅の時系列）を計算し、音が鳴り始めた時点から RMS が最大値（ピーク）に達するまでの時間を求める。全音程にわたるズレの平均値を 60 ms 以内に収めることを目標とする。この数値は、Siedenburg (2019) が 10 楽器 (E12 音程の実験において、AT を  $\leq 60$  ms 操作した際の弁別命中率が約 79%になることから決定した [12]。この水準を「人間が違いに気づきはじめる境界」とみなし、それ未満のズレであれば知覚的に同等とみなせるとし、目標とした。

**スペクトル重心 (SC)：**持続区間（全体の 25～75%）でのスペクトル重心周波数 [Hz]。音の明るさ・硬さの指標であるスペクトル重心が参照音とどれだけズレているかを評価する。音の持続区間（音の長さの 25%～75%の区間）について FFT で周波数スペクトルを求め、各周波数成分をその振幅で重み付けした平均周波数（重心）を計算する。SC は  $SC = \frac{\sum (f_i \times |X(f_i)|)}{\sum |X(f_i)|}$  で求められる。ここで  $f_i$  は周波数、 $X(f_i)$  は振幅である。参照音源と再現音の SC 値の相対差 ( $\frac{|SC_{ref} - SC_{synth}|}{SC_{ref}} \times 100$ ) を音程ごとに算出し、全音程の平均値を求める。全音程にわたる SC 相対差の平均値を 24%以内に収めることを目標とする。この数値は Wun et al. (2014) が複数楽器の平均で SC を  $-24\%$  変化させると弁別率が 75%を超えることを確認したことに基づき決定した [13]。この変化量を「聴感上区別が付き始める閾値」とし、それ未満を目標とした。

**スペクトル変動度 (DSV)：**フレーム間の L2 相対変化量の平均 [%]。音の持続中にスペクトル（音色）がどれだけ揺らぐか、その安定性が参照音とどれだけズレているかを評価する。音声を短いフレームに分割し、連続するフレーム間でのスペクトル（FFT 振幅）の変化量を L2 ノルムで算出し、前フレームのスペクトルで正規化した相対変化量を求める。参照音源と再現音の DSV 値の相対差を音程ごとに算出し、全音程の平均値を求める。全音程にわたる DSV ズレの平均値を 16%以内に収めることを目標とする。この数値は Horner et al. (2004) が 8 楽器を対象とした実験において、スペクトル変動誤差が 16%程度で知覚上の弁別が中程度となることを報告していることに基づき決定した [14]。これを許容できる誤差の上限として採用した。

**奇偶倍音比 (OER)：**奇偶倍音比奇数倍音と偶数倍音のバランス（楽器固有の

音色を決定づける要素、例：クラリネットは奇数倍音優位、フルートは均等）が参照音とどれだけズれているかを評価する。pYIN アルゴリズム [15] で F0（基本周波数）を推定し、 $n \times F0$ （ $n=1,2,3\cdots$ ）の位置にある倍音成分のエネルギーを抽出する。奇数次倍音のエネルギー合計を  $E_{\text{odd}}$ 、偶数次倍音のエネルギー合計を  $E_{\text{even}}$  とし、 $E_{\text{odd}}/(E_{\text{odd}} + E_{\text{even}})$  を算出する。参照音源と再現音の OER 値の相対差を音程ごとに算出し、全音程の平均値を求める。全音程にわたる OER 相対差の平均値を 20%以内に収めることを目標とする。Caclin et al. (2005) がスペクトル不規則性（奇偶倍音比はその指標の一つ）が音色知覚の重要な相関物であることを実験的に確認した [16] こと、McAdams et al. (1995) の知覚次元モデルとも一致するため、相対差 20%以内を目標とした [17]。

音色の知覚には倍音構成・エンベロープ・ノイズ成分など無数の物理的特徴が関わるが、心理学的な多次元尺度構成法（MDS）を用いた聴取実験から、人間が実際に音色を判断する際に利用している主要な次元はごく少数に絞り込めることが示されている。McAdams et al. (1995) は楽器音に対する非類似度判断を MDS 分析し、人間の音色知覚が「立ち上がり時間（時間的特徴）」「スペクトル重心（明るさ）」「スペクトル変動度（音色の時間的揺らぎ）」というごく少数の次元でほぼ説明できることを示した [17]。AT・SC・DSV はこの主要 3 次元に直接対応する指標である。一方、この 3 次元だけでは一部の楽器に残る差（楽器固有の「特異性」）を説明できないことも示されており、Caclin et al. (2005) はその主な要因がスペクトル不規則性（奇数次倍音と偶数次倍音のエネルギーバランスの偏り）であることを実験的に確認した [16]。OER はこの第 4 の次元に対応する指標である。つまり AT・SC・DSV が音色知覚の主要 3 次元、OER がそれだけでは捉えきれない楽器固有の特異性を補う指標として機能しており、この 4 つを組み合わせることで人間が実際に音色の違いとして知覚する範囲をほぼ網羅的に説明できる。そのため、4 項目すべてが目標差以内に収まっていれば、人間が知覚する音色差は小さいと考えられ、参照音源と再現音は知覚上おおむね類似した音色を有すると判断できる。

また、音色の評価に関して、AT・SC・DSV・OER の 4 つの音色特徴量の算出の他にアンケートも同時に行う。作成した回答用フォームを用いた音色評価を実施し、その結果を元に評価を行う。この音色評価は、参加者に作成した音と実際の楽器の音を聞き比べてもらい、目的音をどの程度再現できているかを以下の 5 段階で評価してもらい、その平均値を用いて評価する。集計したものの平均が 3.51 以上の場合、音色が

十分再現されていると判断する。音色の評価では、AT・SC・DSV・OER による音響特徴量の解析結果と、アンケートによる聴取評価の双方を用いて総合的に評価する。

### 1.5.6 プロジェクト中に監視する技術性能指標 TPM

作業分解構造に基づくタスクの消化進捗率を監視する。当初の予定に対して進捗の遅延が 15%を超えた場合には、楽器変更機能などの付加的な機能の優先順位を下げ、コアとなる合奏同期機能の完成を最優先する。

プロジェクト実行中に監視する技術性能指標 TPM を表 1.4 に示す。演奏同期は、光信号受信から演奏開始までの遅延を測定し、100ms を超えた場合には通信方式や処理手順を見直す。楽器演奏は、出力音の周波数を測定し、基準周波数との差を確認する。誤差が大きい場合には、発振周波数や音源生成処理を調整する。LED マトリクス歌詞表示は、演奏タイミングとのズレを測定し、 $\leq 100\text{ms}$  を超える場合には表示更新処理の改善を行う。

これらの TPM を継続的に監視することで、システム全体の品質向上と問題の早期発見を目指す

表 1.4 技術性能指標 TPM

| FBS 要素        | 監視指標             | 理由                      |
|---------------|------------------|-------------------------|
| 演奏同期          | 光信号受信から演奏開始までの遅延 | 同期はシステムの根幹にかかわるため       |
| 楽器演奏          | 参考周波数とのズレ        | 音程の正確さはシステムの品質に直結するため   |
| LED マトリクス歌詞表示 | 演奏タイミングと歌詞表示のズレ  | 視覚情報の同期はシステムの完成度にかかわるため |

## 1.6 資源・調達管理

本開発にあたって必要な機材とその調達方法を表 1.5 に示す。

表 1.5 使用機材一覧

| 使用機材                 | 型番                  | 単価 (円) | 個数  | 調達方法        |
|----------------------|---------------------|--------|-----|-------------|
| マイコンボード              | Arduino UNO R4 WiFi | 0      | 5   | 所有          |
| 抵抗器 330 $\Omega$     | -                   | 6      | -   | 所有          |
| 抵抗器 3300 $\Omega$    | -                   | 1      | -   | 所有          |
| ブレッドボード              | 165401020E          | 0      | 3   | 所有          |
| ジャンパワイヤー             | 165012000E          | 0      | 1 式 | 所有          |
| カラーセンサ               | TCS34725            | 500    | 4   | Amazon にて購入 |
| カラー LED              | OSTA5131A-R/PG/B    | 50     | 2   | 秋月電子通商にて購入  |
| Processing 実行用 PC    | MacBook Air/Pro     | -      | 5 台 | 所有          |
| USB Type-C to C ケーブル | -                   | -      | 5   | 所有          |
| タクトスイッチ              | -                   | -      | 1   | 学科借用        |
| 可変抵抗器                | RK09D117000B        | 60     | 1   | 秋月電子通商にて購入  |
| ルータ                  | -                   | 0      | 1   | 学科借用        |
| 覆いの箱                 | -                   | 0      | 1   | 班員私物        |

## 1.7 リスク管理

本プロジェクトにおいて、光同期の精度が低い、進捗が遅く実装が間に合わないなどのリスクが考えられる。光同期の精度が低い場合、周囲の光によるノイズなどの外部からのノイズを排除するために、光通信全体を仕切りで覆うなどの処置を施す。進捗が遅く実装が間に合わない場合、進捗が十分なメンバーが終わっていない作業のサポートを行い、それでも間に合わない場合は優先順位の低い追加機能の実装を諦めることとする。これらをまとめた、リスク内容と発生可能性並びに影響度、対策を表 1.6 に示す。

表 1.6 リスク管理

| リスク内容         | 発生可能性 | 影響 | 対策                     |
|---------------|-------|----|------------------------|
| 光通信の精度が出ない    | 高     | 大  | センサの感度調整・遮光対策を行う       |
| 遅延が大きく同期がとれない | 中     | 大  | テストを早めに行い早期発見する        |
| 想定より実装に時間がかかる | 中     | 大  | クリティカルパスを優先し、余裕日数を活用する |
| メンバー間で進捗に差が出る | 中     | 中  | 進捗をこまめに共有し、タスクを再分配する   |
| データの紛失        | 低     | 大  | GitHubなどでバージョン管理を行う    |
| 部品が届かない・壊れる   | 低     | 中  | 予備部品を確保しておく            |

## 第 2 章 システムの基本設計

### 2.1 機能一覧

本システムの機能一覧を FBS に基づいて表 2.1 に示す。本システムは、指揮者と複数の演奏者からなるオーケストラ形式の同期演奏を実現するための多様な機能を備えている。システムの根幹となる制御機能は、指揮者用 Arduino に直接取り付けられたスイッチによって管理され、これにより LED の点滅開始と終了、すなわち演奏の開始と停止を即座に行うことが可能である。通信においては、フルカラー LED を用いた独自の光通信プロトコルを採用しており、発光色によって各 Arduino の演奏開始を指示する。なお、再生する楽器音は Web サイトから事前に指定された楽器 ID に従うため、色と楽器音は直接対応しない。点滅間隔の長さによって楽曲のテンポを精密に調整する。また、LED の点滅の強さ、すなわち光量のピーク値を算出することで、各楽器の音量を動的に変化させることができ、柔軟な強弱表現を可能としている。

ユーザーとのインターフェースとして機能する Web サイトからは、テンポ変更や全体音量の調節、演奏者ごとの個別音量の調節に加えて、演奏の開始と停止、楽譜選択、ループ演奏選択といった高度な演奏制御が行える。さらに、ネットワーク環境に問題が発生するなどして Web サイトが利用できない事態を想定し、可変抵抗によるアナログ電圧変化を用いたテンポ調整機能もあわせて実装する。視覚的な演出面では、演奏者側の Arduino に搭載された LED マトリクスのうち、横 8× 縦 8 の範囲を利用して歌詞を一文字ずつ表示する。楽譜情報の送信進捗に合わせた動的なテキスト表示をすることによって、エンターテインメント性における独創性を高めている。

表 2.1 本システムの機能一覧

| 機能名           | 機能概要                                                                                                       |
|---------------|------------------------------------------------------------------------------------------------------------|
| システム起動・停止     | 指揮者用 Arduino に取り付けられた物理スイッチを操作することで、全体の演奏開始および終了を制御する。                                                     |
| 光信号同期通信       | 指揮者側 LED の発光色によって各パートの演奏開始を指示し、点滅間隔の長さに基づいてテンポを調節する。                                                       |
| Web 遠隔制御      | Web ブラウザを介して、BPM の指定、全体音量の調節、楽器パートの変更、楽譜選択、演奏者ごとの個別音量の調節、およびループ演奏の切り替えを指示する。                               |
| 楽器音合成・再生      | 演奏者側 PC の Processing 上で、受信した音高、光量由来の音強、点滅由来の音長情報を基に波形を合成し再生する。                                             |
| 個別音量・表現制御     | LED の点滅の強さ（光量のピーク値）から個別の音量倍率を算出し、動的な音楽表現を実現する。                                                             |
| 歌詞表示演出        | LED マトリクスの横 8× 縦 8 範囲を利用し、カタカナを用いて歌詞を一文字ずつ表示する。輪唱において各パートが異なるタイミングで演奏を開始するため、聴覚のみでは把握しにくい演奏構造を視覚情報として補完する。 |
| アナログ制御（予備）    | ネットワーク環境等により Web 制御が困難な場合、可変抵抗の電圧変化を利用してテンポを可変させる。                                                         |
| 同期確認用 LED(箱外) | 視覚フィードバックおよび遅延計測用に使用する LED、指揮者側 LED と同じ光り方をする。                                                             |

## 2.2 システム構成図

ここで、システム全体の構成図を図 2.1 に示す。

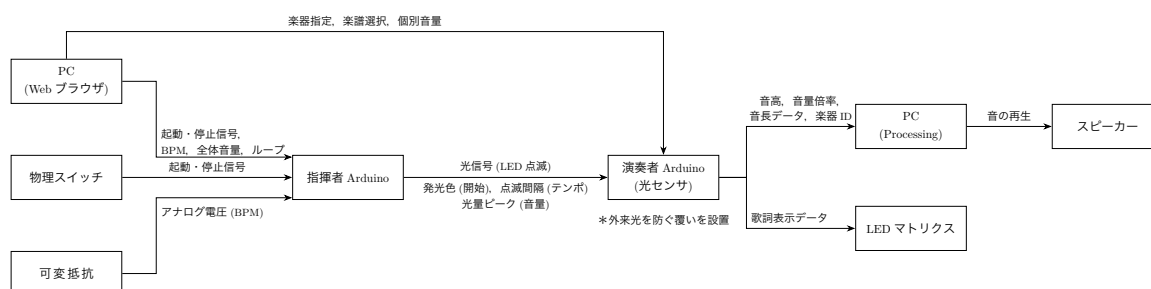


図 2.1 システム構成図

## 2.3 操作インターフェース設計

本システムの操作インターフェースでは、主系統である Web ブラウザ UI と、予備系統である物理的なアナログ入力の方を設計する。

主系統のインターフェースとして、WiFi ネットワーク経由の HTTP 通信を利用したブラウザベースのコントロールパネルを用意する。図 2.2 に Web ブラウザの UI 配置を示す。



## Arduino オーケストラ コントロールパネル

【指揮者】

BPM:

120

送信

全体音量:

MAX

演奏:

開始

停止

ループ:

OFF

【演奏者】

演奏者 1

楽器:

ピアノ ▾

個別音量:

MAX

楽譜:

リズム ▾

送信

演奏者 2

楽器:

ピアノ ▾

個別音量:

MAX

楽譜:

リズム ▾

送信

演奏者 3

楽器:

ピアノ ▾

個別音量:

MAX

楽譜:

リズム ▾

送信

演奏者 4

楽器:

ピアノ ▾

個別音量:

MAX

楽譜:

リズム ▾

送信

全演奏者に一括送信

図 2.2 Web ブラウザの UI 配置

システムを運用するための各種入力フォームが直感的に操作できるよう配置する。具体的には、楽曲のテンポを数値で指定する BPM 入力欄、全体音量を調節するスライダー、演奏の開始・停止ボタン、ループ演奏のトグルスイッチを指揮者セクション

22

に配置する。また、演奏者ごとに楽器の変更、個別音量の調節、楽譜の選択が行える演奏者カードを設ける。入力された各種パラメータは、送信ボタンを押すことで HTTP 通信によって Arduino へ送信される仕組みである。

一方、予備システムのインターフェースとして、ネットワーク障害や PC の不具合等により Web UI が利用不可能な事態を想定し、指揮者用 Arduino に直接接続された可変抵抗を用いた物理的な操作機構を用意する。可変抵抗を操作することによって生じるアナログ電圧の変化を Arduino の A/D コンバータで読み取り、その数値を BPM にマッピングすることで、直感的なテンポ制御を可能とする。これにより、不測の事態においても合奏の続行は担保することができる設計となっている。

## 2.4 状態遷移図

本システムの中心となる指揮者用 Arduino は、外部からの入力に応じて複数の状態を遷移しながら動作を継続する。図 2.3 にその詳細な状態遷移を示す。システムの電源投入および初期設定の完了後、デバイスは「待機・定周期点滅状態」へと移行し、接続要求を待つ。この状態において Web ブラウザから BPM 情報を受信した場合「BPM 更新処理」へと遷移し、内部変数の書き換えが完了した後に再び待機状態へと復帰する。

また、指揮者用 Arduino に搭載された物理スイッチが押下 (ON) されると、システムは「演奏・同期信号送出状態」へと移行し、設定された BPM に基づいた光信号の送出を開始する。演奏中に再度スイッチが押下 (OFF) される、あるいは Web ブラウザから停止命令を受信した場合には、信号送出を停止して初期の待機状態へと遷移する仕組みである。

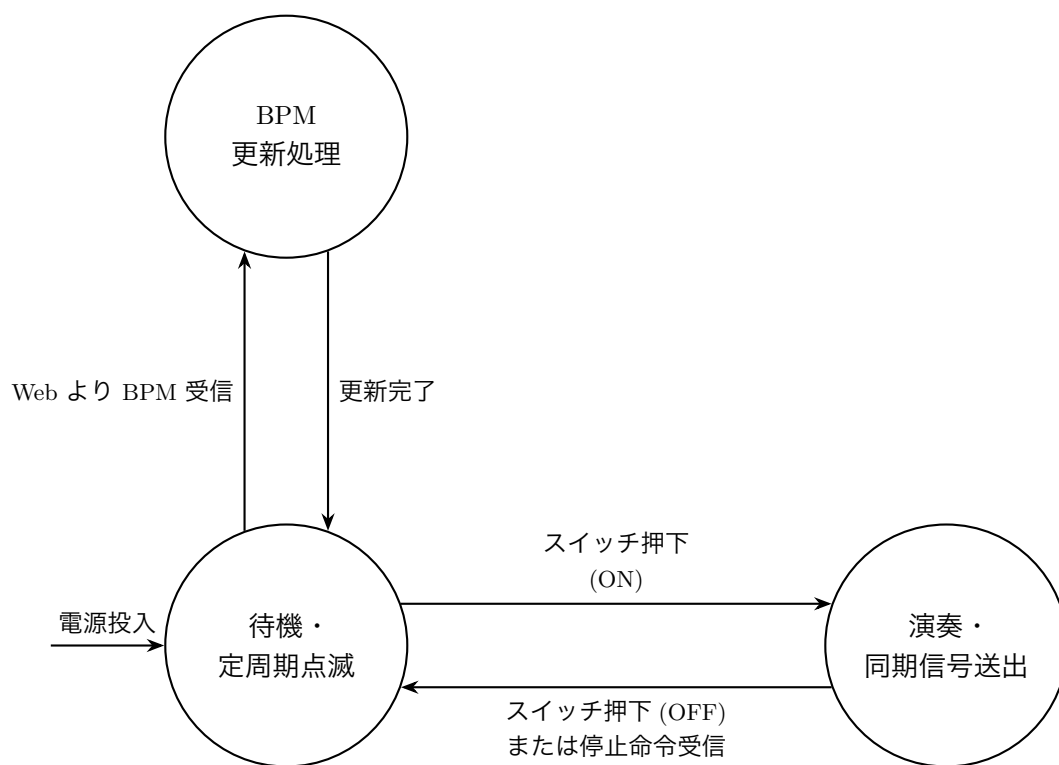


図 2.3 指揮者 Arduino の状態遷移図

## 第 3 章 システムの詳細設計

### 3.1 部品一覧

本システムを構築するために使用する主要なハードウェア部品の一覧を表 3.1 に示す.

表 3.1 本システムの主要部品一覧

| 部品名       | 型番・仕様               | 用途および選定理由                                                                                                                               |
|-----------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| マイクコントローラ | Arduino Uno R4 WiFi | 指揮者側および演奏者側のメイン制御を行う。PC とのシリアル通信や、センサ・LED の高速な制御に必要な処理能力を満たす。                                                                           |
| カラーセンサ    | TCS34725            | 演奏者側デバイスにおいて、指揮者側からの光信号（発光色および光量）を読み取るために使用する。RGB 値に加えて Clear 値（光量そのものの値）を直接出力できる特性を持つため、音量計算用の値を直接測定することができ、処理負荷の軽減が期待できる点が選定の決め手となった。 |
| フルカラー LED | OSTA5131A-R/PG/B    | 指揮者側デバイスにおいて、演奏開始・停止、テンポ、および音量の各情報を、発光色、点滅間隔、光強度という光信号に変換して空間へ送出するために使用する。                                                              |
| LED マトリクス | 8×8 ドット             | 演奏者側デバイスにおいて、楽曲の進行に合わせて歌詞を表示する演出用ディスプレイとして使用する。解像度の制約上、カタカナを一文字ずつ描画する。Arduino Uno R4 WiFi 内蔵のものを利用する。                                   |
| 物理スイッチ    | タクトスイッチ             | 指揮者側デバイスにおいて、システム全体の合奏を物理的に起動および停止させるためのトリガー入力として使用する。                                                                                  |
| 可変抵抗      | ポテンシオメータ            | Web インターフェースがネットワーク障害等で使用できない場合の予備系統として用意する。ツマミの回転によるアナログ電圧変化を BPM 制御に利用する。                                                             |

## 3.2 ハードウェア設計

本システムのハードウェアは、図 3.1 に示すように、独立した指揮者デバイスと演奏者デバイスの 2 種類の筐体で構成される。本ハードウェア構成にした理由は、指揮者の出す LED 同期信号を複数の演奏者が同時に受信し遅延を減少させることを目的としたため、図 3.1 のように構成した。

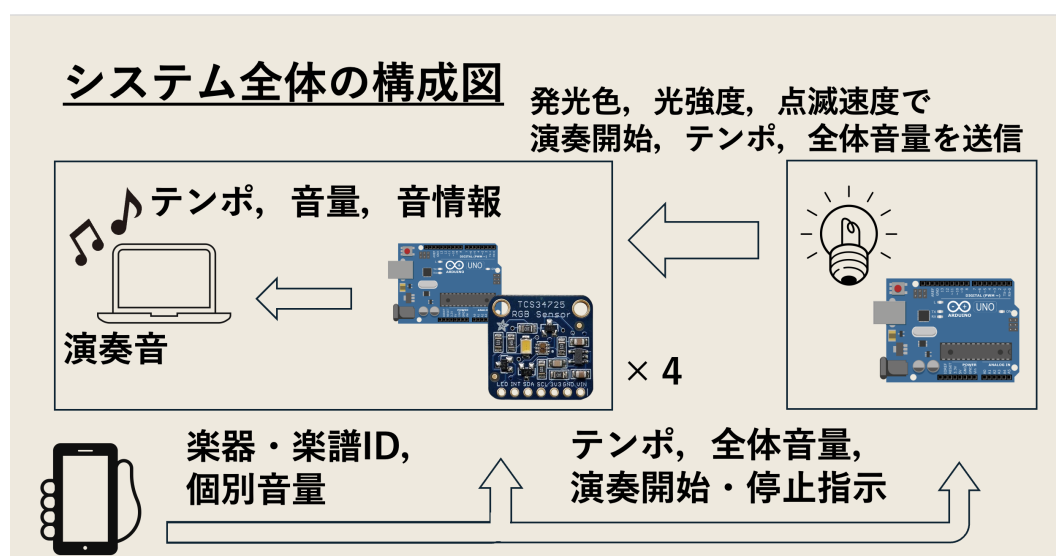


図 3.1 指揮者および演奏者デバイスの構成図

また、本システムの LED 通信部の指揮者，演奏者 Arduino の回路図を図 3.2 に示す。指揮者デバイスでは、フルカラー LED 赤，青，緑のピンを Arduino の D9，D10，D11 ピンへ接続し，PWM 制御によって発光色及び光強度を制御する。演奏者デバイスでは，カラーセンサと I2C 通信を行うため，Arduino で用意されている SDA，SCL，電源，2 つの GND ピンを用いて受光機能を実現する。このとき，電圧はカラーセンサごとの最大電圧に応じて決定する。

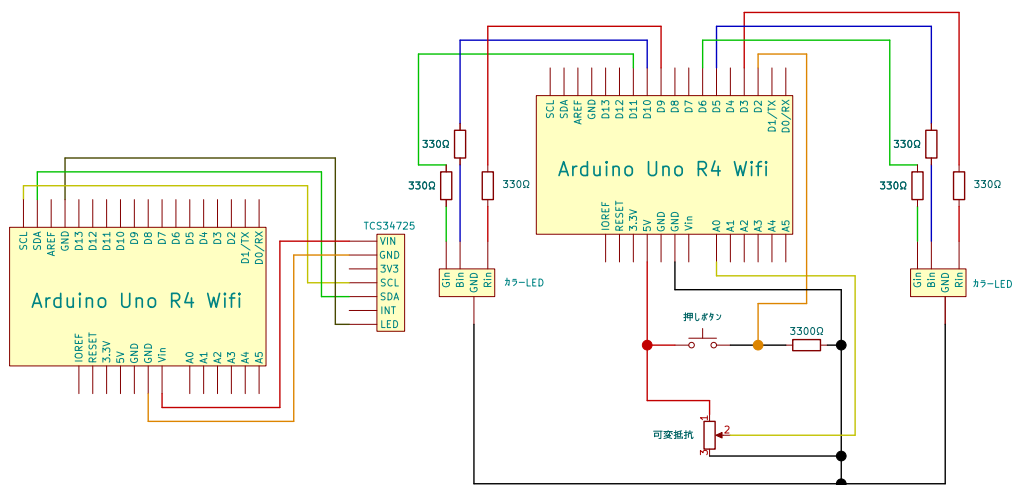


図 3.2 指揮者および演奏者デバイスの回路図

各デバイスの主制御には、内蔵 LED マトリクスを備えた Arduino Uno R4 WiFi を採用し、それぞれの役割に応じた回路で構成する。

指揮者デバイスの回路構成は、システム制御のための入力インターフェースと、信号送出のための出力インターフェースによって形成される。物理的な起動・停止を制御するスイッチは、Arduino の 5V と GND の間に接続し、スイッチと GND の間に D2 ピンを接続する。また、Web UI と同様に機能する可変抵抗は、両端の端子を 5V および GND に、中央の端子を A0 ピンに接続し、分圧された電圧値をテンポ情報として読み取る構成とした。光信号の送出を担う 4 ピン仕様のフルカラー LED は、GND に加え、過電流防止用の 330 Ω の抵抗を介して信号入力端子を D9 (赤)、D11 (緑)、D10 (青) ピンへ接続することで、一つの LED による色彩および光量制御を実現している。

本システムはカラーセンサの色検知精度を向上させるため、センサ周辺を箱で遮光する設計としている。そのため、指揮者 LED が箱の内部に収まり、以下の 2 つの問題が生じる。第一に、利用者が発光状態を視認できず、システムの動作状況を把握しにくい。第二に、テスト時に LED の発光タイミングをカメラで捉えられず、遅延時間の計測が困難である。これらの問題に対応するため、箱の外にも同一の発光制御を行う 2 個目のフルカラー LED を追加する。信号入力端子を D3 (赤)、D5 (青)、D6 (緑) ピンへ 1 個目の LED と同様に 330 Ω の抵抗を介して接続する。

一方、演奏者デバイスの回路構成は、受光および視覚的フィードバックに特化した設計となっている。指揮者側からの光信号を検知するカラーセンサは、I2C 通信インターフェースを利用して接続される。具体的には、モジュールの VIN および GND を Arduino の 5V および GND へ接続し、SDA および SCL ピンをそれぞれの通信専用ピンへと結線している。またモジュールの LED もモジュールに付属している LED が発光するのを防ぐため、GND に接続している。

## 3.3 ソフトウェア設計

### 3.3.1 ファイル構成

本システムのソフトウェアは、指揮者デバイス、演奏者デバイス、および PC 上で動作する音声再生プログラム、Web サイトでの演奏制御プログラムの 4 種類によって構成される。各プログラムの主要ファイルを表 3.2 に示す。

表 3.2 ソフトウェアのファイル構成

| ファイル名               | 動作環境        | 主な処理内容                                        |
|---------------------|-------------|-----------------------------------------------|
| conductor.ino       | 指揮者 Arduino | BPM 取得, LED 発光制御, 演奏開始・停止制御, Web 受信処理を行う。     |
| playerRGBMethod.ino | 演奏者 Arduino | カラーセンサによる光信号受信, 色判定, 音情報のシリアル送信, Web 受信処理を行う。 |
| soundPlayer.pde     | Processing  | Arduino から受信した音情報に基づき, 楽器音の再生および制御を行う。        |
| lyrics.h            | 演奏者 Arduino | LED マトリクスに表示する歌詞データ, 楽譜データを管理する。              |
| index.html          | PC          | 利用者が楽器やテンポ等の演奏パラメータを選択するための操作画面を定義する。         |
| script.js           | PC          | 送信ボタン押下時に設定値を JSON 形式でサーバーへ送信する処理を担う。         |
| server.js           | PC          | Node.js 環境で動作し, ブラウザからの要求を各 Arduino へ中継する。    |

### 3.3.2 通信設計

#### 3.3.2.1 LED 通信

指揮者 Arduino と演奏者 Arduino 間での通信の内容と方法を示す。この区間での通信内容は、表 3.3 のように LED の点滅によるテンポ、点灯時のピーク時の光量による全体音量、LED の発光色による演奏者 Arduino ごとの開始タイミングである。



表 3.3 LED 通信内容

| 送信データ              | LED 表現                         | 受信側処理                                                                                                           |
|--------------------|--------------------------------|-----------------------------------------------------------------------------------------------------------------|
| テンポ (bpm)          | 指揮者の保持する bpm に応じて LED を点滅      | 前回 LED 全体の光量が閾値を超えたときとの時間差で計算する.                                                                                |
| 全体音量               | 音量を光量で制御する.                    | ピーク時の CLEAR 値を CLEAR_MAX_LOW から CLEAR_MAX_HIGH の範囲で 20～100 にリスケーリングし, 個別音量倍率をかけて音量を算出する.                        |
| Arduino ごとの開始タイミング | 特定の LED の色を演奏者の開始タイミングとして設定する. | 紫, 赤, 青, 緑を受光した RGB 値の閾値で判別し, 各色に対応する Arduino を識別する. 自身の担当色を検出した Arduino が, Web サイトから事前に受信した楽器 ID に従って音再生を開始する. |

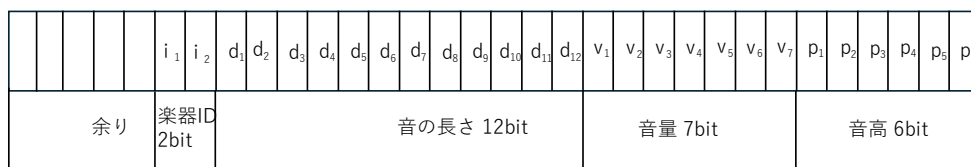
### 3.3.2.2 シリアル通信

演奏者 Arduino と楽器演奏用の Processing 間での通信の内容と方法を示す. 送信内容は, 表 3.4 のように, 楽器変更, 音再生のために音高, 音量, 音の長さ, 楽器 ID を送信する. この区間の通信は Arduino と Processing 側の PC を USB Type-C で繋げ, シリアル通信を行う. シリアル通信の送受信は, データ転送レート bps(baud) を合わせる. 送信は `serial.write(buf,len)` を用いて行う. `Serial.write` は 1 Byte ずつデータを送ることができ, 本送信データは合計 4 Byte 以下であるため, 4 つの 1 Byte 配列で送信する. そのため, 図 3.3 のように, 通信データをビットシフトで非負 32 bit 整数に入れ込み, バイト配列に入れていく. 受信は, `SerialEvent` 関数を用いて 4 Byte 受け取ったら, データを取り出し音を再生する.

表 3.4 送信内容

| 送信データ                 | 送信内容                                 | 送信ビット数 |
|-----------------------|--------------------------------------|--------|
| 音高 (pitch)            | ドレミファソラシドを (0-31) に対応し送信             | 6 bit  |
| 音量 (volume)           | 音量を (0-100) で送信                      | 7 bit  |
| 音の長さ (duration)       | 一音の長さを (0-4095)ms で送信                | 12 bit |
| 楽器 ID(instrumentData) | ピアノ・バイオリン・フルート・パーカッションを (0-3) に対応し送信 | 2 bit  |

uint32\_t data



送信データ uint8\_t buf[4]

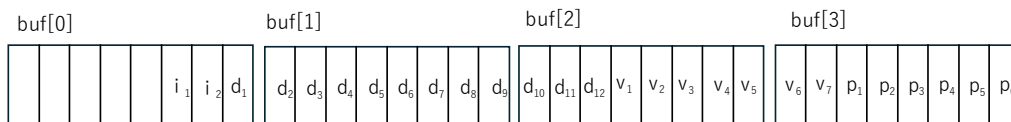


図 3.3 Byte 配列での送信方法

### 3.3.3 LED マトリクスへの歌詞表示

エンターテインメントとして演奏者 Arduino に現在の再生している音に対応した歌詞を表示させる。データは演奏者 Arduino が保持し、表示内容は図 3.4 のように Arduino の LED マトリクスへカタカナを表示させる。表示タイミングは全体の演奏の遅延を考慮して、演奏者 Arduino から Processing へ音情報を送信した後に歌詞を表示させる。データとしては、表 3.5 のように各文字ごとの配列と曲全体の歌詞配列を持つ。このとき、歌詞データは楽譜情報と同じ要素数を持ち、同じインデックスを共有して使用する。それぞれの文字は、定数で 32 ビットの符号なし整数型であり、16 進数の 3 要素で LED マトリクスの 96 bit を表現している。マトリクスをコード内で使用するために必要な関数などを表 3.6 に示す。LED マトリクス用のヘッダファイルと楽譜

情報の入った lyrics ヘッドファイルをインクルードし、オブジェクトを作成する。  
セットアップ関数内で Serial.begin を呼び出し、matrix.loadFrame 関数で表示する。

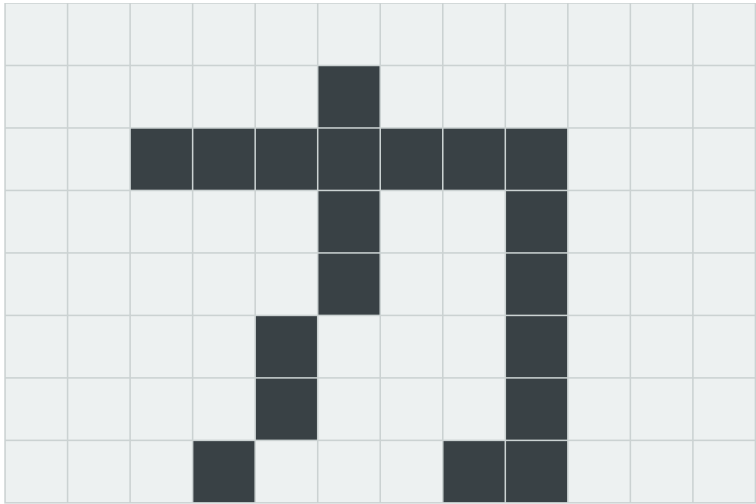


図 3.4 LED マトリクスに 1 文字ずつ表示

表 3.5 歌詞データ

| 型               | 配列名      | 要素内容       | データ                            |
|-----------------|----------|------------|--------------------------------|
| const uint32_t  | ka[3]    | カタカナの「カ」   | 0x201f, 0xc0240240, 0x4404408c |
| const uint32_t  | e[3]     | カタカナの「エ」   | 0x1f004, 0x400400, 0x400403f8  |
| const uint32_t* | lyrics[] | カエルの歌全体の歌詞 | ka, e, ru,..                   |

表 3.6 LED マトリクスを使用するためのコード

| コード                             | 場所        | 内容                       |
|---------------------------------|-----------|--------------------------|
| #include "Arduino_LED_Matrix.h" | コード先頭     | LED マトリクスを使用するためのヘッダファイル |
| #include "lyrics.h"             | コード先頭     | 歌詞データを格納したヘッダファイル        |
| ArduinoLEDMatrix matrix         | コード先頭     | オブジェクトを作成                |
| matrix.begin()                  | setup 関数内 | LED Matrix を開始           |
| matrix.loadFrame(lyrics[index]) | loop 関数内  | 表示させたい文字配列を引数で指定し表示する。   |

### 3.3.4 音生成・再生

楽器音は Processing のオーディオライブラリ Minim を使用し、各楽器の音色特性に応じて、波形、倍音、ノイズ成分、エンベロープを組み合わせて生成する。

ピアノ・バイオリン・フルートの音色生成には加算合成を用いる。加算合成とは、複数のサイン波（基音および倍音）を重ね合わせることで任意の音色を表現する手法であり、出力波形  $y(t)$  は式 (3.1) で表される。

$$y(t) = \sum_{n=1}^N A_n \sin(2\pi n f_0 t) \quad (3.1)$$

ここで、 $f_0$  は基音周波数、 $n$  は倍音次数、 $A_n$  は各倍音の振幅である。 $A_n$  の組み合わせを楽器ごとにすることで音色の違いを表現する。パーカッションはノイズ成分と低周波サイン波を組み合わせて打撃音を生成する。

各楽器の倍音構成  $A_n$  は、実際の楽器音の録音データに対して FFT（高速フーリエ変換）を適用し、得られた周波数スペクトルから基音および各倍音成分の振幅を読み取ることで決定する。また、録音データ全体の振幅を解析し、Attack, Decay, Sustain, Release の各パラメータを取得する。取得した ADSR パラメータは、振幅の時間変化として波形生成に反映する。

### 3.3.5 受光機能

#### 3.3.5.1 点滅間隔計測

LED の点滅間隔を計測することで、演奏テンポの取得を行う。clear 値が閾値を超えた時刻を記録し、連続する 2 回の検出時刻の差を点滅間隔として算出する。この値を演奏タイミング制御へ利用する。

図 3.5 に点滅間隔計測の概要を示す。

## 受光機能(点滅間隔の計測)

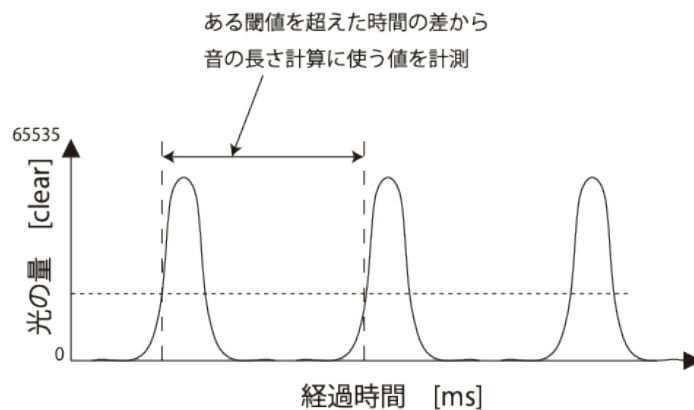


図 3.5 点滅間隔計測の概要

### 3.3.5.2 光量ピーク検出

受光した光量値の最大値を利用することで、演奏音量の制御を行う。受光した光が強いほど大きな音量、弱いほど小さな音量となるように設定することで、光強度に応じた演奏表現を可能とする。

図 3.6 に光量最大値計測の概要を示す。

## 受光機能(光量最大値の計測)

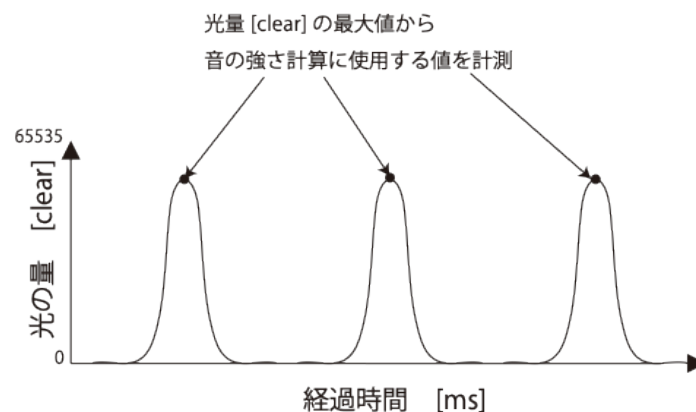


図 3.6 光量最大値計測の概要

### 3.3.5.3 発光色の識別

カラーセンサから取得した RGB 値を用いて、発光色の識別を行う。本システムでは赤・緑・青・紫の 4 色を使用し、各色に異なる Arduino を割り当てる。演奏者 Arduino は割り当てられた色を検出したら、演奏を開始する。一度自色を検出した後は、以降の点滅に対して色判定を行わず点滅検出のみで楽譜を進行させる。色の判定は RGB 値そのものではなく、図 3.7 に示すように各成分の割合 ( $R/(R+G+B)$  など) が事前測定により決定した閾値を上回るか下回るかによって行う。具体的には、R 成分の割合が閾値以上かつ G・B 成分の割合が閾値以下の場合を赤と判定するなど、各色に対応する RGB 割合の条件を図 3.8 に示す方法で事前測定により決定し、色識別を行う。

## 割合導出と閾値チェック

$$\begin{aligned} \text{R 値の割合} &= \frac{\text{R 値}}{\text{R 値} + \text{G 値} + \text{B 値}} & \text{R 値の割合} \geq 50 & \leftarrow \text{予め色判定の割合を決定} \\ \text{G 値の割合} &= \frac{\text{G 値}}{\text{R 値} + \text{G 値} + \text{B 値}} & \text{G 値の割合} \leq 20 & \rightarrow \text{割合閾値を満たしたため 赤色と判定} \\ \text{B 値の割合} &= \frac{\text{B 値}}{\text{R 値} + \text{G 値} + \text{B 値}} & \text{B 値の割合} \leq 20 & \end{aligned}$$

図 3.7 RGB 割合の計算方法

## 赤色であることを検出する場合

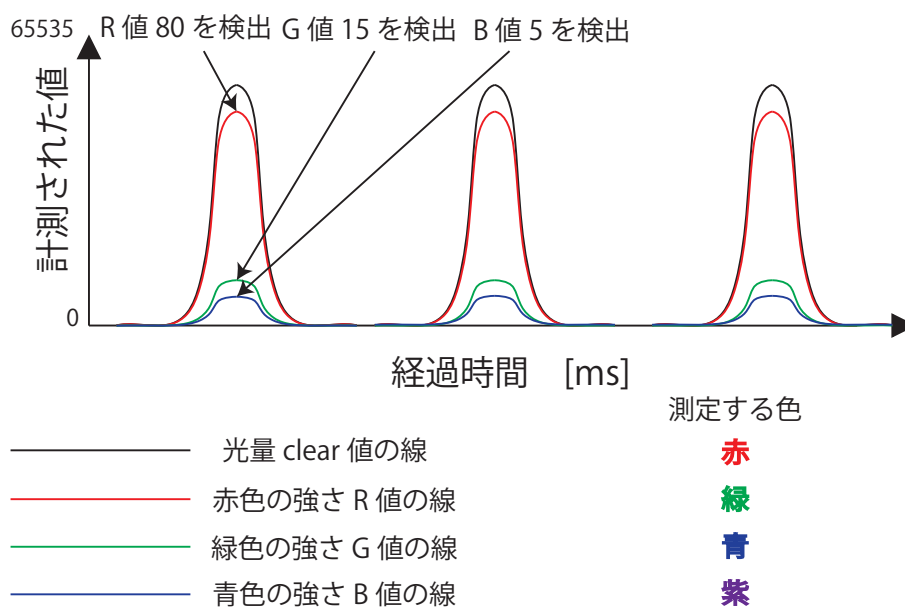


図 3.8 RGB 割合の計測方法

## 3.4 スイッチによる入力制御の詳細設計

本節では、指揮者デバイスにおける物理入力、すなわち演奏の起動・停止を担うタクトスイッチと、予備系統としてテンポを制御する可変抵抗について、その具体的な入力制御の方法を述べる。これらの入力は、図 2.1 に示すシステム構成のうち、指揮者 Arduino への入力インターフェースに該当する。以降の設計は、検証用に実装した指揮者デバイスの入力制御プログラム (LED\_switch.ino) の動作を基にしている。

### 3.4.1 入力ピンの割り当てと初期設定

指揮者デバイスでは、タクトスイッチ、可変抵抗、および同期用 LED をそれぞれ独立したピンに割り当てる。各入力の接続および設定方法を表 3.7 に示す。

表 3.7 指揮者デバイスの入力ピン割り当てと設定

| 入力要素        | ピン         | 設定および接続方法                                                                                                                                            |
|-------------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| タクトスイッチ     | D2         | INPUT_PULLUP に設定し、内部プルアップ抵抗を有効化する。スイッチの他端を GND へ接続し、外付け抵抗を不要とするとともにノイズによる誤検出を抑制する。                                                                   |
| 可変抵抗        | A0         | INPUT に設定し、A/D コンバータでアナログ電圧を読み取る。3 端子の可変抵抗（ポテンショメータ）を用い、両端の端子を 5V・GND に、中央端子（ワイパ）を A0 へ接続することで、可変抵抗自体が分圧器として機能し、ツマミ位置に応じた電圧をテンポ情報として取得する（3.4.3 項参照）。 |
| 同期用 LED     | D9,D11,D10 | OUTPUT に設定し、analogWrite による PWM 制御で発光の ON/OFF および光強度を制御する。                                                                                           |
| 同期用 LED（箱外） | D3,D5,D6   | 箱内 LED と同一の制御信号を出力する。利用者への視覚フィードバックおよびテスト時の遅延計測を目的として追加する。                                                                                           |



タクトスイッチを INPUT\_PULLUP とすることで、スイッチが押されていないときは入力が HIGH、押されたときに GND へ落ちて LOW となる。プログラム上では、押下を真として扱うために読み取り値を論理反転し、「押下時に true」となる状態変数として保持する。

### 3.4.2 可変抵抗によるテンポ入力

可変抵抗から読み取ったアナログ値を、演奏テンポ（BPM）および同期用 LED の点滅間隔へ変換する。A/D コンバータは入力電圧を 0～1023 の整数値として出力する。この入力値  $v$  を、演奏に用いるテンポ範囲 30～240 BPM へ対応させる。変換後の BPM  $B$  は式 (3.2) で与えられる。

$$B = 30 + \frac{240 - 30}{1023} v \quad (3.2)$$

得られた BPM から、同期用 LED が 1 回点滅する周期（点灯と消灯を合わせた時間） $T_{pulse}$  [ms] を式 (3.3) により算出する。本設計では、1 拍を 2 分割した 8 分音符の間隔で点滅させる。

$$T_{pulse} = \frac{60000}{B \times 2} \quad (3.3)$$

ここで、1 回の点滅における点灯時間は、テンポによらず短い固定値  $T_{on}$ （本設計では 20 ms）とし、残りの時間  $T_{pulse} - T_{on}$  を消灯時間に充てる。このように点灯時間を短縮することで、発光が鋭いパルス状となり、受光側のカラーセンサが点滅（テンポ）を検出しやすくなる利点がある。

式 (3.2)・式 (3.3) により、可変抵抗のツマミ位置に応じて LED の点滅間隔が連続的に変化し、ネットワークを介さずに直感的なテンポ制御を可能とする。可変抵抗の値はメインループ内で常時読み取られ、演奏中のリアルタイムなテンポ変更にも対応する。

### 3.4.3 可変抵抗による分圧の原理

前項のテンポ入力を成立させるには、ツマミの操作に応じて A0 ピンの電圧が連続的に変化する必要がある。本設計では、図 3.9 に示すように、3 端子の可変抵抗（ポテンショメータ）を分圧器として用いる。

3 端子の可変抵抗は、抵抗体の両端（端子 1・端子 3）と、抵抗体上を摺動するワイ

パ（端子 2）からなる。ワイパによって抵抗体は、端子 1 側の抵抗  $R_a$  と端子 3 側の抵抗  $R_b$  の 2 つに分割される。ここで、端子 1 を電源電圧  $V_{cc}$  (5 V)、端子 3 を GND、ワイパを A0 ピンへ接続すると、A0 ピンの電圧  $V_{A0}$  は式 (3.4) で与えられる。

$$V_{A0} = V_{cc} \cdot \frac{R_b}{R_a + R_b} \quad (3.4)$$

ツマミを回すとワイパ位置が移動し、 $R_a$  と  $R_b$  の比が変化するため、式 (3.4) に従って  $V_{A0}$  が 0 V から  $V_{cc}$  まで連続的に変化する。このように、可変抵抗 1 つで分圧器が構成されるため、外付けの固定抵抗を別途追加する必要はない。この電圧変化を A/D コンバータで 0～1023 の値として読み取り、式 (3.2) により BPM へ対応させることで、テンポの可変制御を実現する。

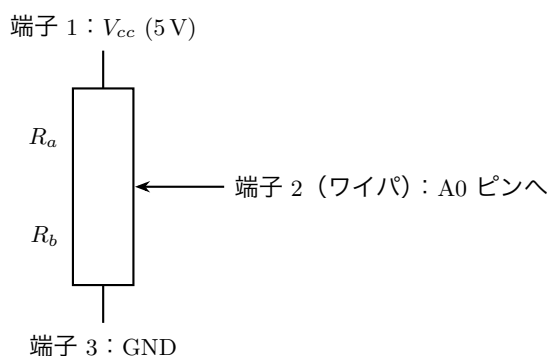


図 3.9 3 端子可変抵抗（ポテンショメータ）による分圧

### 3.4.4 タクトスイッチによる起動・停止制御

タクトスイッチは、押すたびに演奏の開始・停止が交互に切り替わるトグル動作とする。ここで、スイッチを「押している間 ON」とする単純な読み取りではなく、「押された瞬間」のみを一度だけ検出してシステム状態を反転させる方式を採用する。これは、1 回の押下で複数回の切り替えが発生することを防ぐためである。

#### 3.4.4.1 押下エッジの検出

押された瞬間を検出するため、直前のループでのスイッチ状態  $s_{prev}$  と現在のスイッチ状態  $s_{now}$  の 2 つを保持する。そして、 $s_{prev}$  が非押下 (false) かつ  $s_{now}$  が押下 (true) へ変化した遷移を「押下の立ち上がりエッジ」として検出する。この立ち上がりエッジが検出されたときにのみ、システム全体の ON/OFF 状態  $S_{sys}$  を反転さ

せる。ループの末尾で  $s_{prev} \leftarrow s_{now}$  と更新することで、次回ループでの変化検出に備える。

#### 3.4.4.2 チャタリング（バウンス）対策

機械式スイッチでは、押下・開放の瞬間に接点が短時間振動し、意図しない複数回の ON/OFF が発生するチャタリング（バウンス）が生じる。これを抑制するため、状態を切り替えた時刻  $t_{last}$  を記録し、前回の切り替えから一定時間  $T_{deb}$ （本設計では定数として 50 ms を与える）が経過していない間は、新たな押下エッジを無効とする時間的なフィルタリングを行う。すなわち、押下エッジの検出条件は式 (3.5) を満たす場合に限る。

$$\neg s_{prev} \wedge s_{now} \wedge (t_{now} - t_{last} > T_{deb}) \quad (3.5)$$

この時間判定は、押下エッジの検出条件と同一の条件式の中で評価する実装とした。条件が成立した場合のみ  $S_{sys}$  を反転し、 $t_{last}$  を現在時刻  $t_{now}$  で更新する。ここで、経過時間の判定にはマイコンの起動からの経過ミリ秒を返す関数（millis）を用い、処理を止めて待つブロッキング遅延を使わずに実装する。これにより、待機中も可変抵抗の読み取りや LED 点滅などの他処理を継続したまま、1 回の物理的な押下を確実に 1 回の状態遷移へ対応させることができる。

#### 3.4.4.3 システム状態に応じた点滅制御

システム状態  $S_{sys}$  が ON（true）のときのみ、同期用 LED を点滅させる。このとき、点灯状態では前回の切り替えからの経過時間が点灯時間  $T_{on}$  を超えた時点で消灯へ、消灯状態では経過時間が  $T_{pulse} - T_{on}$  を超えた時点で点灯へ切り替える。すなわち、短い点灯と相対的に長い消灯を交互に繰り返すことで、鋭いパルス状の発光を  $T_{pulse}$  の周期で送出する。点灯時の光強度は PWM のデューティ比により設定し、全体音量に対応する光量として送出する。 $S_{sys}$  が OFF（false）のときは LED を消灯し、点滅状態を初期化する。スイッチの読み取りと状態判定を高速に繰り返すため、メインループは短い周期で実行する構成とする。

### 3.4.5 入力制御の処理フロー

以上のタクトスイッチおよび可変抵抗による入力制御の流れを，図 3.10 に示す．本処理はメインループ内で繰り返し実行され，可変抵抗値の読み取りによるテンポ更新と，スイッチ押下エッジの検出によるシステム状態の切り替え，状態に応じた LED 点滅制御を担う．

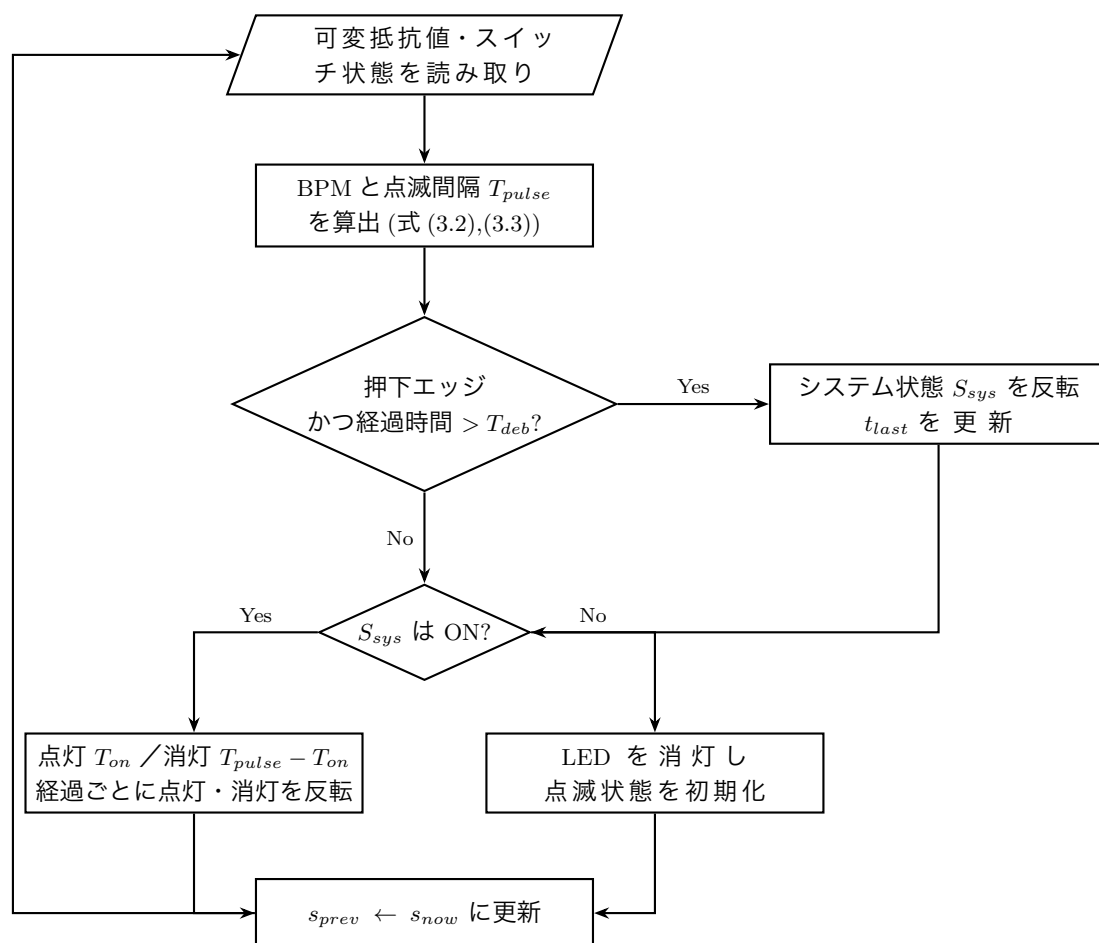


図 3.10 タクトスイッチ・可変抵抗による入力制御の処理フロー

このように，本システムの起動・停止およびテンポ制御は，押下エッジ検出とチャタリング対策を備えたトグル方式のスイッチ入力，ならびに可変抵抗のアナログ値を BPM へ線形変換するテンポ入力によって実現する．

### 3.4.6 オブジェクト設計 (クラス図)

全体で使用するクラスを図 3.11 に示す。使用するクラスは、楽器再生の Processing で使用する楽器クラスで構成されている。各楽器クラスは `Instrument` インタフェースを実装しており、`PianoInst` (ピアノ)、`FluteInst` (フルート)、`ViolinInstrument` (バイオリン)、`SnareInstrument` (スネアドラム) の 4 種類が定義されている。各クラスは `playNoteFromData()` 関数から楽器 ID に応じて呼び出され、音高・音量・音長の情報をもとに、それぞれの音合成アルゴリズムで音を生成・出力する役割を持っている。指揮者デバイス (`conductor.ino`) では、BPM や演奏状態などの変数を保持し、LED 発光制御・Web 受信・BPM 表示を制御する役割を持っている。演奏者デバイス (`playerRGBMethod.ino`) では、カラーセンサからの RGB 値や音符番号などの変数を保持し、LED 同期による色判定・音情報の生成・シリアル送信を制御する役割を持っている。

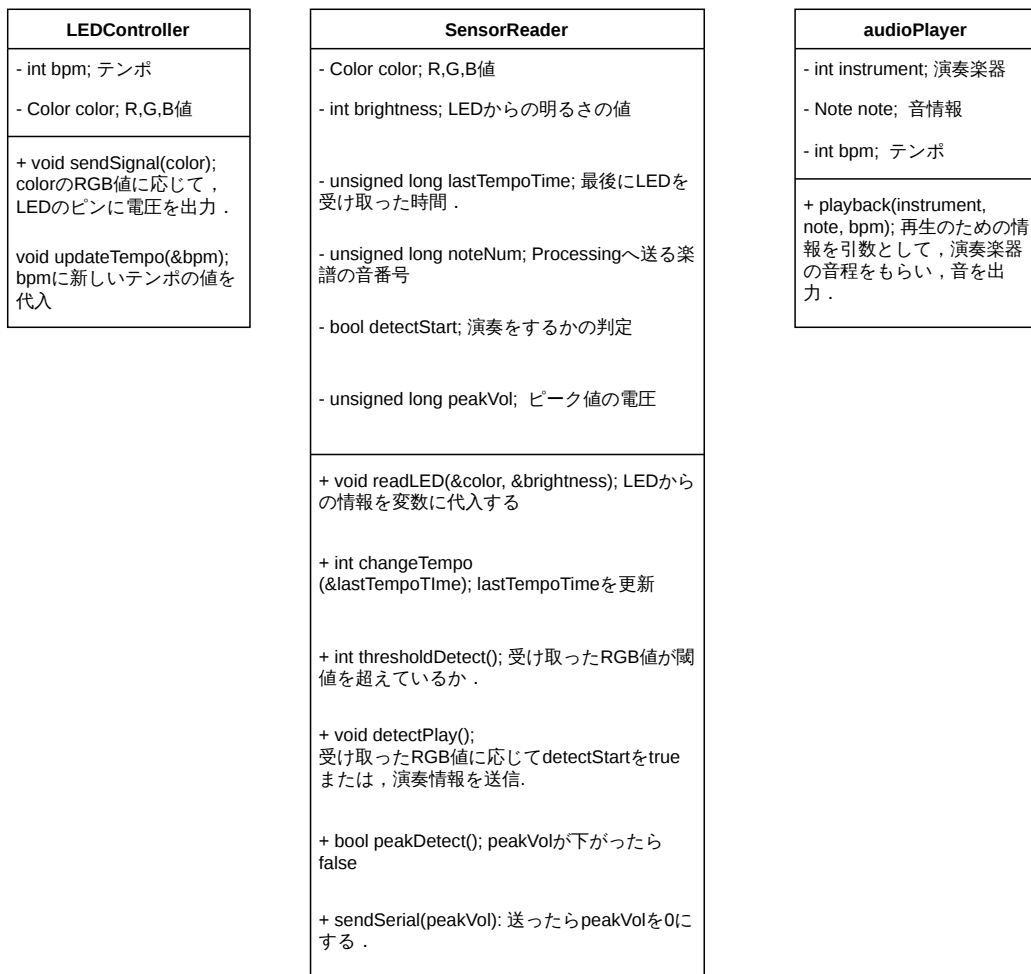


図 3.11 役割ごとのクラス図

### 3.4.7 関数設計

表 3.8 LED 同期関連の関数設計

| 関数名                     | 引数                  | 戻り値        | 処理                                                                       |
|-------------------------|---------------------|------------|--------------------------------------------------------------------------|
| applyLED(c, level)      | c : 点灯色, level : 輝度 | void       | フルカラー LED を指定した色・輝度で点灯させる. (conductor.ino)                               |
| colorForPulse(pulseIdx) | pulseIdx : パルス番号    | LightColor | パルス番号に対応する点灯色を返す. (conductor.ino)                                        |
| detectColorID(r, g, b)  | r, g, b : RGB 値     | int        | 受信した色情報が自分の担当色と一致するか判定し, 色 ID を返す. 一致しない場合は-1 を返す. (playerRGBMethod.ino) |

表 3.9 シリアル通信関連の関数設計

| 関数名            | 引数                | 戻り値  | 処理                                                               |
|----------------|-------------------|------|------------------------------------------------------------------|
| sendNote()     | なし                | void | 音高・音量・音長・楽器 ID を 32bit にパックしてシリアル通信で送信する. (playerRGBMethod.ino)  |
| serialEvent(p) | p : Serial オブジェクト | void | 受信した 4 バイトをシフト演算で分解し, 音高・音量・音長・楽器 ID を変数へ格納する. (soundPlayer.pde) |

表 3.10 LED マトリクス関連の関数設計

| 関数名                               | 引数                                                   | 戻り値  | 処理                                                |
|-----------------------------------|------------------------------------------------------|------|---------------------------------------------------|
| showBPM(bpm)                      | bpm : BPM 値                                          | void | BPM の値を 3 桁の数字として LED マトリクスに表示する. (conductor.ino) |
| drawDigit(bitmap, digit, xOffset) | bitmap : 描画バッファ, digit : 表示する数字, xOffset : X 方向オフセット | void | LED マトリクスの描画バッファに 1 桁の数字を書き込む. (conductor.ino)    |

表 3.11 音再生関連の関数設計

| 関数名                                   | 引数                                        | 戻り値  | 処理                                                 |
|---------------------------------------|-------------------------------------------|------|----------------------------------------------------|
| playNoteFrom-Data(pi, vol, dur, inst) | pi : 音高, vol : 音量, dur : 音長, inst : 楽器 ID | void | 受信した音情報を基に, 楽器 ID に応じた音色で音を再生する. (soundPlayer.pde) |

表 3.12 Web 通信関連の関数設計

| 関数名                     | 引数                           | 戻り値  | 処理                                                                                           |
|-------------------------|------------------------------|------|----------------------------------------------------------------------------------------------|
| handleWebRequest()      | なし                           | void | Web サーバーからの HTTP リクエストを受信し, BPM・音量・演奏開始停止等のパラメータを反映する. (conductor.ino / playerRGBMethod.ino) |
| sendHeartbeat()         | なし                           | void | サーバーへ定期的に登録リクエストを送信し, 接続状態を維持する. (conductor.ino / playerRGBMethod.ino)                       |
| tryReconnect()          | なし                           | void | 待機中にサーバーへの再登録を試み, 成功した場合は Web サーバーを起動する. (conductor.ino / playerRGBMethod.ino)               |
| getWebParam(query, key) | query : クエリ文字列, key : パラメータ名 | int  | クエリ文字列から指定キーの値を取り出す. 存在しない場合は-1 を返す. (conductor.ino / playerRGBMethod.ino)                   |
| enterStandaloneMode()   | なし                           | void | WiFi への接続またはサーバーへの登録に失敗した場合に, WiFi 無しの単体動作モードへ移行する. (conductor.ino)                          |

## 3.5 処理フローの設計

図 3.12 に本システム全体のシーケンス図を示す. 本システムは指揮者 Arduino, 演奏者 Arduino, Processing, Web サイトの 4 つで構成される. Web サイトから BPM, 全体音量, ループ, 起動・停止信号を指揮者 Arduino が受信し, LED による光信号として演奏者 Arduino へ送信する. また, 楽器, 楽譜, 個別音量は Web サイトから演奏



者 Arduino へ直接送信される。演奏者 Arduino は光信号を受信・解析し、Processing へ演奏情報を送信することで音の再生および歌詞表示を行う。なお、光信号を受信できない場合は待機状態を維持し、演奏を開始しない。また、Web サイトから設定情報を受信できない場合は設定変更は行われない。

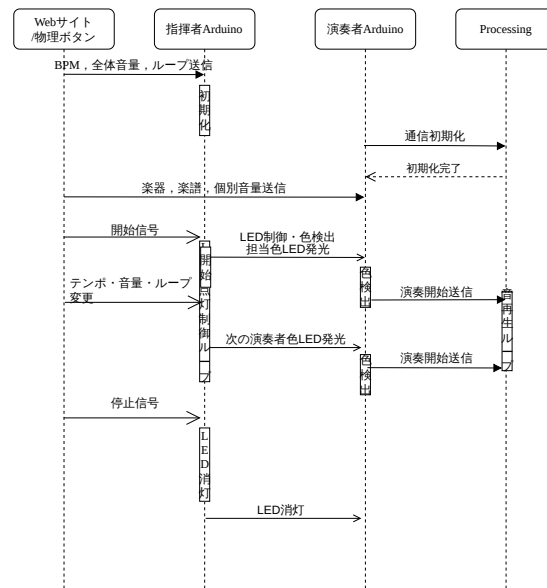


図 3.12 全体の処理のシーケンス図

また、各構成要素間のやり取りの詳細について、図 3.13、図 3.14、図 3.15 に演奏者、指揮者、Web のそれぞれのフローチャートを示す。演奏者は、起動後に Processing との通信を初期化し、担当楽器および担当色を確認する。次に、Web サイトから HTTP 通信によって楽器、楽譜、個別音量の情報を受信する。その後、カラーセンサによる待機状態へ移行し、自身の担当色を検出した場合に Processing へ演奏開始信号を送信し、音の再生を行う。また、演奏開始後に 3000 ms 以上 LED 点滅を検知できなかった場合は、演奏インデックスおよびフラグを初期化してカラーセンサ待機状態へリセットする。停止ボタンが押下されると、Processing との通信を終了し、処理を停止する。

指揮者側では、Web サイトとの通信確立後、BPM、全体音量、ループ、起動・停止信号の情報を受信する。受信した情報をもとに LED の点灯制御を行い、LED を通じて各演奏者の Arduino へ担当楽器のタイミングを共有する。演奏中にテンポ、音量、ループの変更が行われた場合は設定を更新し、停止信号を受信した時点で演奏を停止して LED を消灯する。輪唱の実装では、各パートの演奏開始タイミングを一定パル

ス数 (OFFSET\_PULSES\_PER\_PART) だけずらすことで、パートごとに異なるタイミングで演奏が開始される仕組みとなっている。

Web サイトは、起動後、ユーザが演奏者ごとに楽器、楽譜、個別音量を設定するとともに、BPM、全体音量、ループを設定し、演奏開始ボタンを押下することで指揮者および演奏者へ演奏情報を送信する。演奏中はテンポ、音量、ループの変更の有無を監視し、変更があった場合は随時情報を更新する。演奏停止ボタンが押下された時点で処理を終了する。

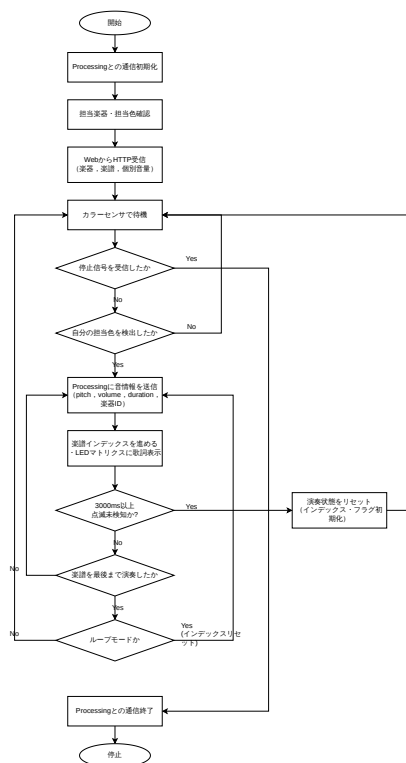


図 3.13 演奏者側の全体のフローチャート

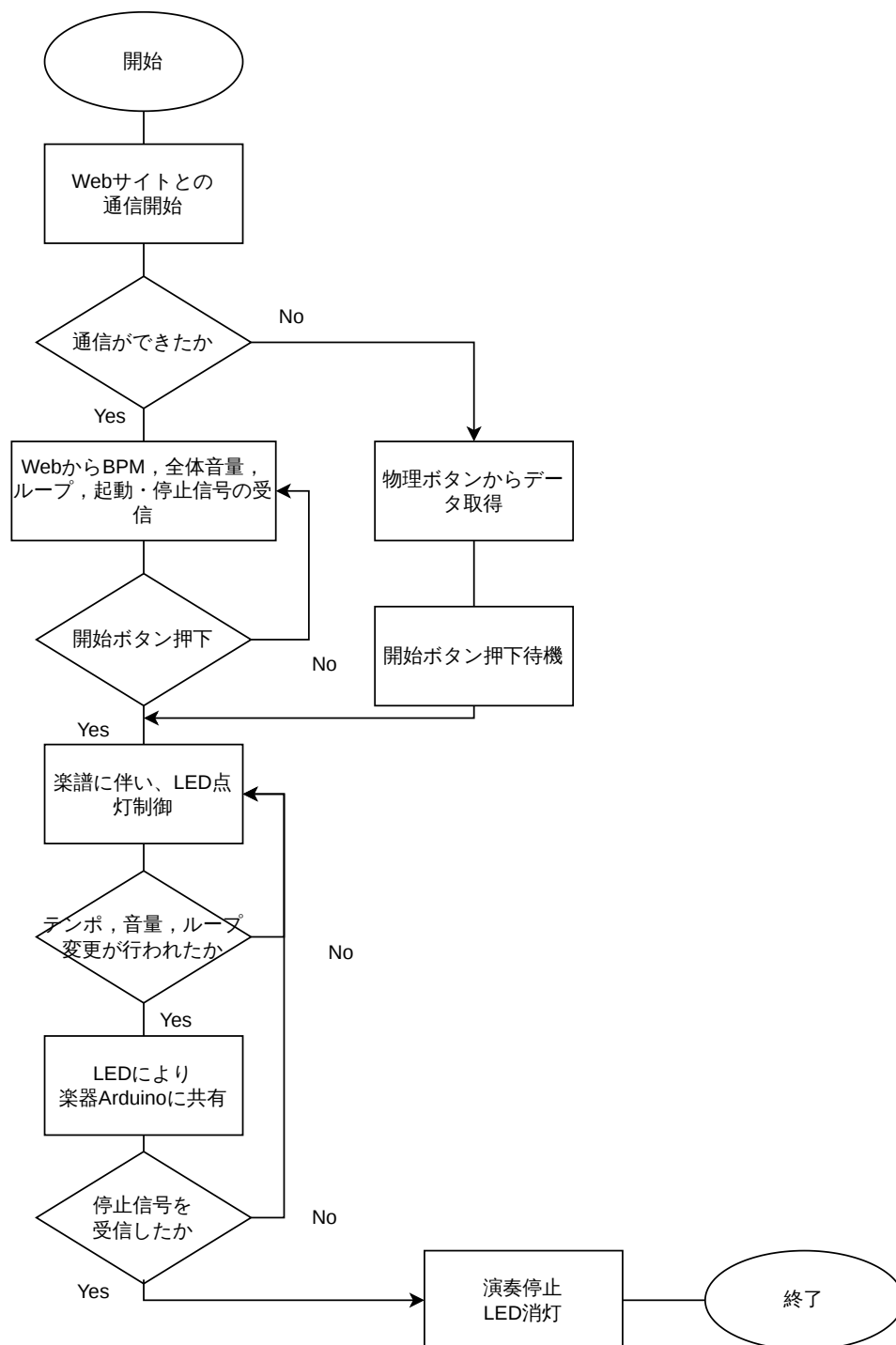


図 3.14 指揮者側の全体のフローチャート

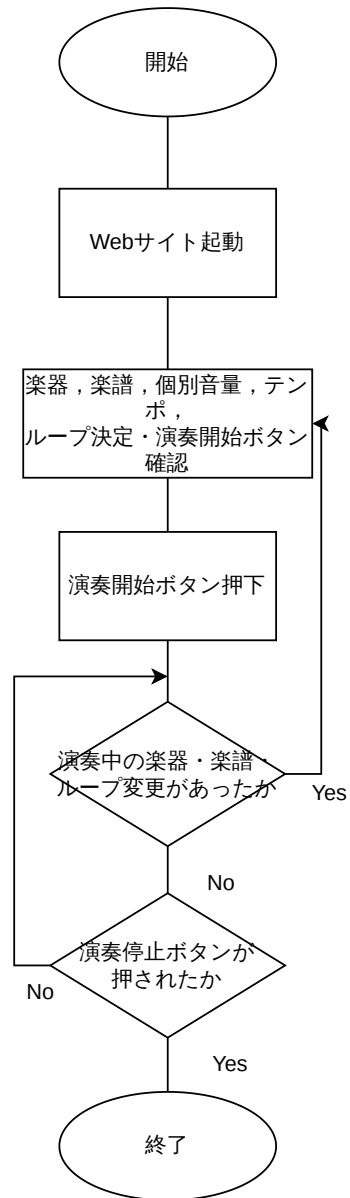


図 3.15 Web 関連の全体のフローチャート

## 3.6 テストの設計

### 3.6.1 システムテストの設計

#### (1) 光と音の発生時間差

スマートフォンまたはカメラのハイスピード撮影機能（240 fps 等）を使用し、指揮者の LED と演奏者 PC のスピーカーが同一画面内に収まるよう動画を撮影する。撮影した動画を PC 上の動画編集ソフトに読み込み、LED が発光したフレームから、スピーカーから音が出たフレームまでのフレーム数を計測し、時間差を算出する。

#### (2) 楽器間の同期ズレ

(1) と同様に、ハイスピード撮影を用いて複数の演奏者 PC のスピーカーを同一画面内に収めて撮影する。撮影した動画から各演奏者の発音フレームを特定し、演奏者間のフレーム差を時間に換算して同期ズレを算出する。

#### (3) Web サイト送信～演奏反映までの遅延

以下の 2 種類の方法を併用して計測する。

##### ・ハイスピード動画撮影による計測：

ハイスピード撮影機能を用いて、Web サイト画面と演奏者 PC のスピーカーが同一画面内に収まるよう動画を撮影する。撮影した動画から、送信が実行されたフレームから音が変わったフレームまでのフレーム数を計測し、遅延時間を算出する。

##### ・ログによるシステム計測：

Web サイト側で送信が実行された瞬間の UNIX タイムスタンプ（ミリ秒単位）をログに記録する。同様に、演奏側プログラムにおいて、受信データの解析およびテンポ変数等の書き換え処理が完了した瞬間のタイムスタンプを記録する。両タイムスタンプの差分を遅延時間として算出する。なお、PC 間で時計が同期していない場合は NTP による時刻同期を行うか、同一 PC 上でテストを実施する。

### 3.6.2 結合テストの設計

#### (1) 光同期のパケット損失率

LED を一定回数（2000 回）発光させ、受信側の光センサが正しく検知できた回数および検知できなかった回数を目視により計数する。発光回数（総パケット数）に対する検知失敗回数（損失パケット数）の割合をパケット損失率（IPLR）として算出する。

#### (2) Web サイトのパケット損失率

Web サイトからテンポ変更や楽器指定などの制御コマンドを一定回数送信し、送信ログと受信ログを出力する。両ログを比較し、総パケット数に対する損失パケット数の割合（パケット損失率／IPLR）を算出する。

### 3.6.3 単体テストの設計

#### (1) 音色の再現性

再現対象となる実際の楽器音の録音データと、本システムから出力された再現音の録音データをそれぞれ用意する。比較の前処理として、両者の音量を RMS 正規化により統一し、pYIN アルゴリズムで推定した F0 を用いて音程が一致していることを確認する。その後、両者の音声波形データから、立ち上がり時間（AT）、スペクトル重心（SC）、スペクトル変動度（DSV）、奇偶倍音比（OER）の 4 つの音色特徴量を算出する。AT は RMS エンベロープのピーク到達時間、SC は持続区間（25～75%）のスペクトル重心周波数、DSV はフレーム間 L2 相対変化量の平均、OER は pYIN で推定した F0 を用いた  $E_{\text{odd}}/(E_{\text{odd}} + E_{\text{even}})$  として算出する。実際の楽器音と再現音の各特徴量を比較し、それぞれの目標差以内であることを確認する。本テストは各楽器・音程ごとに実施し、全結果の平均を取り評価する。

また、音色の評価に関して、AT・SC・DSV・OER の 4 つの音色特徴量の算出の他にアンケートも同時に行う。作成した回答用フォームを用いた音色評価を実施し、その結果を元に評価を行う。この音色評価は、参加者に作成した音と実際の楽器の音を聞き比べてもらい、目的音をどの程度再現できているかを以下の 5 段階で評価してもらう形式とする。

- 5, 似ている
- 4, どちらかといえば似ている
- 3, どちらとも言えない
- 2, どちらかといえば似ていない
- 1, 似ていない

この 5 段階評価の平均値を用いて評価する。集計したものの平均が 3.51 以上の場合、同意と解釈して良いという先行研究 [18] に基づき、集計結果の平均値が 3.51 以上の場合に十分に音色を再現できたと評価する。

アンケートフォーム内には音色評価の他に、年代、音楽経験の有無、演奏環境についての質問も含まれる。年齢を収集する理由は、回答者の聴覚特性の違いが音色評価に影響を与える可能性があるためである。

加齢に伴い高周波数帯域を中心とした聴覚特性の変化が生じることが示されており [19]、これが楽器音のような複雑な音の音質・音色知覚や主観評価に影響を及ぼすと考えられる。したがって、評価データの偏りを防ぎ、年代ごとの知覚傾向を正しく分析するために年齢の項目を設けた。

音楽経験の有無を収集する理由は、音楽に関する知識や訓練経験の違いが音色評価に影響を与える可能性があるためである。音楽経験者は楽器音に接する機会が多く、音色の特徴や違いを識別する能力が高いと考えられる。一方で、音楽経験のない回答者は一般的な聴取者としての評価を行うため、両者の評価傾向に差が生じる可能性がある。したがって、音楽経験の有無による評価傾向の違いを分析するために当該項目を設けた。

視聴環境を収集する理由は、再生機器の特性の違いが音色評価に影響を与える可能性があるためである。イヤホン、ヘッドホン、PC スピーカ、スマートフォン内蔵スピーカなどの再生機器は周波数特性や音圧特性が異なり、同一の音源であっても聴取者が知覚する音質や音色が変化する場合がある。したがって、視聴環境による評価結果への影響を分析し、必要に応じて評価結果の解釈に反映するために当該項目を設けた。

## 参考文献

- [1] 神田竜, 岩野成利, 片寄晴弘, 複数のモバイルデバイスを九十九神と見立てるインタラクティブメディアアート: 九 i 九 i 九, 情報処理学会論文誌, 53 巻, 3 号, p1061-1068.
- [2] 亀川 徹, 4-3 演奏時における音声遅延の検知限とその影響, 映像情報メディア学会誌, 2015, 69 巻, 5 号, p. 408-411.
- [3] 総務省, 情報通信白書令和 2 年版, 総務省, 2020, <https://www.soumu.go.jp/johotsusintokei/whitepaper/ja/r02/pdf/index.html>, 2026 年 4 月 21 日参照.
- [4] Fada Pan, Li Zhang, Yuhong Ou, and Xinni Zhang, "Audiovisual integration effect on music emotion: Behavioral and physiological evidence," PLoS ONE, Vol. 14, No. 5, p. e0217040, 2019.
- [5] 新 誠一, 計測制御のための無線通信技術の現状と展望, 計測と制御, 2009, 48 巻, 7 号, p. 537-541.
- [6] 若森 和彦, 光無線通信と画像伝送, 画像電子学会誌, 2011, 40 巻, 1 号, p. 257-263.
- [7] International Telecommunication Union, [Recommendation] Relative timing of sound and vision for broadcasting (Recommendation ITU-R BT.1359-1), ITU Radiocommunication Assembly, 1998.
- [8] A. D. Brown, G. C. Stecker, D. J. Tollin, "The Precedence Effect in Sound Localization," Journal of the Association for Research in Otolaryngology, Vol. 16, No. 1, pp. 1-28, 2015.
- [9] R. B. Miller, "Response time in man-computer conversational transactions," in Proceedings of the AFIPS Fall Joint Computer Conference, Vol. 33, pp. 267-277, 1968.
- [10] International Telecommunication Union, "Network performance objectives for IP-based services," ITU-T Recommendation Y.1541, 2011.
- [11] S. McAdams, S. Winsberg, S. Donnadieu, G. De Soete, J. Krimphoff, "Perceptual scaling of synthesized musical timbres: Common dimensions, specificities, and latent subject classes," Psychological Research, Vol. 58, No. 3, pp. 177-192, 1995.
- [12] K. Siedenburg, "Timbral and dynamic differentiability of musical instrument tones," The Journal of the Acoustical Society of America, Vol. 145, No. 2, pp. 1078-1087, 2019.
- [13] C. Wun, A. Horner, C. Wu, "Timbre discrimination of musical instruments in melody," Journal of the Audio Engineering Society, Vol. 62, No. 9, pp. 575-583, 2014.



- [14] A. Horner, J. Beauchamp, R. So, "Detection of random spectral variation in musical instrument sounds," *The Journal of the Acoustical Society of America*, Vol. 116, No. 3, pp. 1800-1810, 2004.
- [15] M. Mauch and S. Dixon, "pYIN: A fundamental frequency estimator using probabilistic threshold distributions," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 659-663, 2014.
- [16] A. Caclin, S. McAdams, B. K. Smith, S. Winsberg, "Acoustic correlates of timbre space dimensions: A confirmatory study using synthetic tones," *The Journal of the Acoustical Society of America*, Vol. 118, No. 1, pp. 471-482, 2005.
- [17] S. McAdams, S. Winsberg, S. Donnadieu, G. De Soete, J. Krimphoff, "Perceptual scaling of synthesized musical timbres: Common dimensions, specificities, and latent subject classes," *Psychological Research*, Vol. 58, No. 3, pp. 177-192, 1995.
- [18] J. R. Lindner, N. J. Lindner, "Interpreting Likert-type Scales, Summated Scales, Unidimensional Scales, and Attitudinal Scales: I neither Agree nor Disagree, Likert or Not," *Advancements in Agricultural Development*, Vol. 5, No. 2, 2024.
- [19] 入野 俊夫, 加齢性難聴の模擬による聴覚特性理解へのアプローチ, *日本音響学会誌*, 2025, 81 巻, 5 号, p. 352-359.